

Faculté des Sciences et Techniques
Département d'Informatique

N° d'ordre : 01 / 26

RAPPORT

de Projet de Fin d'Études

Présenté en Vue de l'Obtention de
Licence en Sciences et Techniques

Spécialité : Ingénierie de Données et Développement Logiciel

MISE EN PLACE D'UN PIPELINE MLOPS SIMPLE POUR LA PRÉDICTION DE LA RÉUSSITE ACADÉMIQUE

Réalisé par :

Fatima EL AYACHI

Encadrés par :

Soutenu le 23 juin 2026 devant le jury composé de :

Pr. 	Président
Pr. 	Examineur
Mme. 	Invitée
Pr. 	Encadrant

Année universitaire : 2025 / 2026

Dédicaces

À mes très chers parents

Les mots me manquent pour traduire la profondeur de mon affection et de ma reconnaissance à votre égard. Vous avez été, dès mes premiers pas, les architectes de mon éducation et les gardiens de mes rêves.

À ma très chère Mère, havre de paix et de tendresse, dont l'amour bienveillant et les prières quotidiennes ont guidé mes pas dans les moments de doute.

À mon noble Père, modèle de droiture et de sagesse, qui m'a enseigné la valeur de l'effort, la noblesse de la persévérance et le goût de l'excellence.

Votre soutien indéfectible et vos sacrifices consentis dans l'ombre ont tracé la voie de ma réussite. Puisse ce travail, fruit de vos enseignements, honorer vos espérances et témoigner de ma piété filiale et de mon éternelle gratitude.

À mon cher frère et mes chères sœurs

Merci pour votre présence réconfortante, votre soutien moral et votre affection sincère. Vos encouragements permanents et votre confiance ont été d'une aide précieuse durant toutes les étapes de cette aventure académique. Vous demeurez ma force, mon équilibre et une source inestimable de bonheur.

À ma chère tante

Je tiens à vous adresser mes sincères remerciements pour votre bienveillance, votre soutien chaleureux et vos encouragements constants. Votre présence à mes côtés a toujours été pour moi un précieux réconfort.

À ma famille et à tous ceux qui m'ont soutenue

Je tiens à vous exprimer ma profonde gratitude pour votre bienveillance, votre soutien inconditionnel et votre présence rassurante tout au long de mon parcours. Vos encouragements constants et votre affection sincère ont constitué pour moi une véritable source de réconfort et de motivation.

Remerciements

Louanges à Allah, le Tout-Puissant, le Clément et le Miséricordieux, qui m'a accordé la force, la patience et la persévérance nécessaires pour mener à bien ce modeste travail. Rien de tout cela n'aurait été possible sans Sa miséricorde et Sa guidance.

J'adresse mes sincères remerciements à l'ensemble de mes professeurs pour la qualité de leur enseignement, leur engagement et la richesse de leurs conseils. Leur savoir et leur accompagnement ont constitué une source constante d'inspiration tout au long de mon parcours universitaire.

J'exprime également ma profonde gratitude à mon encadrant, M. ADIL SOUFI, pour son accompagnement, ses conseils précieux et sa disponibilité tout au long de ce projet. Son encadrement a été déterminant dans la réussite de ce travail.

Ma profonde gratitude revient à mes très chers parents. Leur amour inconditionnel, leurs sacrifices et leur soutien indéfectible ont été ma plus grande force et ma principale motivation. Ce travail est autant le fruit de leurs efforts que des miens.

Je remercie également mon frère et mes sœurs pour leur présence, leur soutien moral et leurs encouragements constants, qui m'ont permis d'avancer avec confiance.

Enfin, j'exprime ma reconnaissance à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet.

Résumé

La prédiction de la réussite scolaire constitue un enjeu important pour les établissements d'enseignement, car elle permet d'identifier précocement les étudiants susceptibles de rencontrer des difficultés académiques et de mettre en place des actions d'accompagnement adaptées.

Ce travail présente la conception et la mise en œuvre d'une solution de prédiction de la réussite scolaire basée sur les principes du Machine Learning et du MLOps. Le projet s'appuie sur le jeu de données *Student Performance Dataset* et suit les différentes étapes du cycle de vie d'un projet de Machine Learning, depuis l'analyse exploratoire des données jusqu'au déploiement du modèle en production.

Une phase de préparation des données a été réalisée afin de nettoyer, transformer et analyser les informations disponibles. Deux modèles de classification supervisée, la Régression Logistique et l'Arbre de Décision, ont ensuite été entraînés et évalués à l'aide de plusieurs métriques de performance telles que l'Accuracy, le F1-Score et la matrice de confusion. Les expériences ont été suivies et comparées grâce à la plateforme MLflow, permettant la gestion centralisée des paramètres, des métriques et des versions des modèles via le *Model Registry*.

Dans une démarche d'industrialisation inspirée des pratiques MLOps, le modèle retenu a été déployé sous la forme d'une API REST développée avec FastAPI. Une interface web interactive réalisée avec Streamlit a également été mise en place afin de permettre la saisie des caractéristiques d'un étudiant et la génération de prédictions en temps réel.

Les résultats obtenus démontrent l'intérêt de l'intégration des pratiques MLOps dans le développement de solutions basées sur le Machine Learning, en améliorant la traçabilité, la reproductibilité et le déploiement des modèles tout en proposant un outil concret d'aide à la prédiction de la réussite scolaire.

Mots-clés : Machine Learning, MLOps, MLflow, Régression Logistique, Arbre de Décision, FastAPI, Streamlit, Prédiction de la réussite scolaire.

Liste des acronymes

Acronyme	Signification
PFE	Projet de Fin d'Études
IA	Intelligence Artificielle
ML	Machine Learning (Apprentissage Automatique)
MLOps	Machine Learning Operations
API	Application Programming Interface
EDA	Exploratory Data Analysis (Analyse Exploratoire des Données)
EDM	Educational Data Mining
EdTech	Educational Technology
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
UI	User Interface (Interface Utilisateur)
UX	User Experience (Expérience Utilisateur)
DT	Decision Tree (Arbre de Décision)
LREG	Logistic Regression (Régression Logistique)
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
F1-score	Mesure de performance (Moyenne harmonique entre la précision et le rappel)
TP	True Positive (Vrai Positif)
TN	True Negative (Vrai Négatif)
FP	False Positive (Faux Positif)
FN	False Negative (Faux Négatif)
PaaS	Platform as a Service
SaaS	Software as a Service
HTTP	HyperText Transfer Protocol
XAI	Explainable Artificial Intelligence (IA Explicable)
LIME	Local Interpretable Model-agnostic Explanations
SHAP	SHapley Additive exPlanations

SOMMAIRE

Dédicaces	1
Remerciement	2
Résumé	3
Liste des acronymes	4
Liste des Figures	9
Liste des Tableaux	11
Introduction Générale	12
Contexte et définition du projet	13
Problématique	14
Objectifs du projet	14
Objectif général	15
Objectifs spécifiques	15
Méthodologie adoptée	15
1  ÉTAT DE L'ART ET CONCEPTS FONDAMENTAUX	17
1.1 Intelligence artificielle et Machine Learning	18
1.1.1 Définition de l'intelligence artificielle	18
1.1.2 Définition du Machine Learning	18
1.1.3 Types de Machine Learning et classification	19
1.2 Machine Learning appliqué à l'éducation (EdTech)	21
1.2.1 Analyse des données éducatives	21

1.2.2	Détection précoce des étudiants à risque	21
1.3	Vers le MLOps : Limites du ML traditionnel et cycle de vie . . .	21
1.3.1	Limites des projets ML traditionnels	22
1.3.2	Besoin du MLOps	22
1.3.3	Cycle de vie d'un projet MLOps	22
1.4	Spécifications et choix de la pile technologique	23
1.4.1	Gestion du cycle de vie des modèles : MLflow	23
1.4.2	Architecture API : FastAPI	23
1.4.3	Interface utilisateur : Streamlit	24
1.4.4	Isolation de l'environnement : Docker	24
1.4.5	Plateformes d'hébergement PaaS : Hugging Face Spaces & Streamlit Cloud	24
1.5	Conclusion	25
2	 PRÉPARATION DES DONNÉES ET ANALYSE EXPLORA- TOIRE	26
2.1	Présentation et description générale du dataset	27
2.1.1	Source des données	27
2.1.2	Description générale du dataset	27
2.2	Analyse exploratoire des données (EDA)	27
2.2.1	Analyse univariée : Distribution de la variable cible . . .	28
2.2.2	Analyse bivariée : Facteurs comportementaux de réussite	28
2.2.3	Étude des corrélations initiales	29
2.3	Prétraitement des données et feature engineering	30
2.3.1	Nettoyage des données et gestion des valeurs manquantes	30
2.3.2	Encodage des variables catégorielles	30
2.3.3	Sélection finale des caractéristiques	31
2.4	Labellisation et partitionnement du dataset	32
2.4.1	Transformation et labellisation de la variable cible . . .	32
2.4.2	Séparation des données d'entraînement et de test	33
2.5	Conclusion	33

3	 MODÉLISATION MACHINE LEARNING ET GESTION DU CYCLE DE VIE DES MODÈLES	35
3.0.1	Automatisation du pipeline et reproductibilité des données	36
3.1	Présentation et entraînement des modèles de classification . . .	36
3.1.1	Stratégie d'entraînement et de partitionnement	36
3.1.2	Régression logistique	37
3.1.3	Arbre de décision	37
3.2	Métriques d'évaluation	37
3.3	Résultats et synthèse comparative des modèles	38
3.3.1	Résultats et métriques de la Régression Logistique (Mo- dèle Champion)	39
3.3.2	Résultats et métriques de l'Arbre de Décision	40
3.3.3	Synthèse comparative des performances	40
3.4	Tracking des expériences avec MLflow	41
3.4.1	Présentation de MLflow	41
3.4.2	Utilisation de MLflow Autolog	41
3.4.3	Suivi des métriques et paramètres	41
3.4.4	Comparaison des expériences et alertes de non-convergence	43
3.5	Gestion du Model Registry	43
3.5.1	Enregistrement des modèles	44
3.5.2	Sélection et promotion du modèle champion	44
3.6	Conclusion	45
4	 INDUSTRIALISATION ET DÉPLOIEMENT DE LA SOLU- TION	46
4.1	Architecture logicielle globale de la solution	47
4.2	Développement du Backend avec FastAPI	48
4.2.1	Structure du projet	48
4.2.2	Création des endpoints et validation des schémas	49
4.3	Tests fonctionnels de l'API et validation des contrats	50
4.4	Conteneurisation de l'API avec Docker	52
4.4.1	Dockerfile	52
4.4.2	Construction et exécution de l'image	53

4.5	Déploiement de l'API sur Hugging Face Spaces	54
4.6	Conception de la couche cliente avec Streamlit	55
4.6.1	Interface utilisateur et formulaire de saisie	56
4.6.2	Affichage des résultats	56
4.7	Déploiement du Frontend sur Streamlit Community Cloud . . .	57
4.8	Connexion entre Streamlit et FastAPI	57
4.9	Tests finaux de la solution	58
4.10	Monitoring, Observabilité et Workflow MLOps	59
4.11	Conclusion	61
CONCLUSION GÉNÉRALE		62
	Bilan du projet	62
	Limites de la solution	63
	Perspectives	63

Liste des Figures

1.1	Classification des différentes branches du Machine Learning. . .	20
1.2	Cycle de vie itératif d'un projet de Machine Learning Operations (MLOps).	23
2.1	Distribution empirique des notes finales G3 et ajustement de densité	28
2.2	Impact du temps d'étude hebdomadaire sur G3	29
2.3	Dispersion des notes en fonction des absences	29
2.4	Matrice de corrélation de Pearson sur les variables numériques initiales	29
2.5	Matrice de corrélation globale après encodage des variables catégorielles	31
3.1	Matrice de confusion graphique obtenue sur l'ensemble de validation pour le modèle champion	39
3.2	Courbe ROC et mesure de l'AUC	40
3.3	Courbe Précision-Rappel	40
3.4	Courbes de performance et analyse de sensibilité du classifieur champion	40
3.5	Tableau de bord de l'interface locale MLflow UI et historique complet des runs	42
3.6	Détails des exécutions et métadonnées associées au projet de tracking sur l'interface MLflow	43
3.7	Vue globale des modèles enregistrés au sein du registre centralisé MLflow	44

3.8	Statut du registre pour la Régression Logistique affichant la version championne promue en Production	45
4.1	Architecture globale du système MLOps pour la prédiction des résultats scolaires	47
4.2	Structure de l'arborescence du projet et versioning sur le dépôt distant GitHub	48
4.3	Structure de l'API FastAPI : Logique de chargement du modèle (gauche) et fonction d'inférence (droite)	49
4.4	Validation fonctionnelle de l'endpoint de prédiction et analyse de la réponse via Postman	50
4.5	Interface Swagger UI de l'API listant les endpoints et les schémas de données associés	51
4.6	Exécution d'un test d'inférence réussi avec retour de la charge utile JSON via l'interface Swagger	52
4.7	Script de configuration Dockerfile pour la conteneurisation et le déploiement de l'API FastAPI.	53
4.8	Registre local des images Docker et build de la solution d'inférence	53
4.9	Supervision du conteneur actif et télémétrie des logs via Docker Desktop	54
4.10	Tableau de bord utilisateur Hugging Face et statut du Space de l'API	55
4.11	Structure du référentiel de fichiers et des artefacts au sein du Space Hugging Face	55
4.12	Formulaire Streamlit de saisie des caractéristiques étudiantes . .	56
4.13	Affichage de l'analyse prédictive finale sur l'interface de production Streamlit	57
4.14	Flux de communication entre Streamlit et FastAPI	58
4.15	Dashboard Sentry affichant la capture d'une erreur en temps réel.	60
4.16	Monitoring de disponibilité (Uptime 100%) via UptimeRobot. .	60

Liste des Tableaux

2.1	 Dictionnaire des caractéristiques — Version	
	FINALE	32

Introduction Générale

Dans le domaine de l'éducation, la question de la réussite scolaire reste un défi majeur pour les établissements d'enseignement. Chaque année, de nombreux étudiants rencontrent des difficultés scolaires qui les conduisent à l'échec, souvent sans que les enseignants aient pu intervenir à temps. Pourtant, les données nécessaires pour anticiper ces situations existent déjà : notes intermédiaires, absences, temps d'étude, environnement familial.

C'est dans ce contexte que s'inscrit ce projet de fin d'études. L'objectif principal est de concevoir une solution basée sur le Machine Learning permettant de prédire le risque d'échec scolaire d'un étudiant à partir de ses caractéristiques académiques et personnelles. Pour aller plus loin qu'un simple prototype de recherche, nous avons adopté une approche MLOps afin de rendre cette solution traçable, reproductible et déployée en production sous forme d'application web accessible.

Ce rapport présente les différentes étapes du projet, depuis l'analyse des données jusqu'au déploiement final de la solution.

Contexte et définition du projet

Le secteur éducatif est aujourd'hui confronté à des défis liés à la réussite académique des étudiants. L'échec scolaire constitue un problème important qui peut avoir de grandes conséquences sur le parcours académique et professionnel des apprenants. Plusieurs facteurs influencent ces performances, y compris le temps d'étude, l'absentéisme, l'environnement familial, les habitudes de travail ainsi que les conditions socio-économiques.

Avec l'explosion du volume de données dans le domaine éducatif, les méthodes traditionnelles d'analyse deviennent rapidement limitées et difficiles à exploiter manuellement. Dans ce contexte, les établissements éducatifs ont besoin de solutions intelligentes capables d'automatiser l'analyse des données et de fournir des prédictions fiables sur le risque d'échec scolaire [1, 2].

L'apprentissage automatique se présente alors comme une solution appropriée, car il permet de construire des modèles capables d'apprendre à partir des données historiques afin de prédire la performance future des élèves. Cette approche favorise une identification précoce des étudiants en difficulté et contribue à améliorer la prise de décision pédagogique [3, 4].

Par ailleurs, les projets modernes d'intelligence artificielle nécessitent l'adoption de pratiques MLOps afin de garantir :

- **Reproductibilité** des expériences scientifiques ;
- **Suivi rigoureux** du cycle de vie des modèles ;
- **Gestion centralisée** des versions (données, code et paramètres) ;
- **Déploiement automatisé et fiable** des applications ;
- **Mise en production sécurisée** des modèles d'intelligence artificielle [5, 6, 7].

Dans ce cadre, ce projet propose de concevoir et de mettre en œuvre une application intelligente pour prédire les risques d'échec scolaire basée sur une approche globale de MLOps. La solution développée couvre l'ensemble du cycle de vie du modèle, depuis l'analyse exploratoire des données (EDA) jusqu'au

déploiement en production. Elle inclut le prétraitement des données, l'entraînement des modèles de Machine Learning, le suivi des expériences avec MLflow, le développement d'une API REST avec FastAPI, la création d'une interface utilisateur avec Streamlit, ainsi que la conteneurisation avec Docker et le déploiement cloud de l'infrastructure [8, 9, 10, 11, 12].

❓ Problématique

Bien que la disponibilité des données académiques augmente, les établissements d'enseignement continuent de rencontrer des difficultés à identifier précocement les étudiants à risque d'échec.

De plus, l'exploitation des données éducatives à grande échelle nécessite des outils capables d'automatiser l'analyse des informations et de produire des prévisions fiables pour soutenir la prise de décisions pédagogiques. Cependant, après la phase de développement des modèles, de nombreux projets d'apprentissage font face à des défis liés à la répétition des expériences, à la gestion des versions, au suivi du cycle de vie des modèles, ainsi qu'à leur déploiement dans des environnements de production.

Dans ce contexte, il devient nécessaire de concevoir une solution qui intègre à la fois les techniques d'apprentissage automatique pour prédire les risques d'échec scolaire et les pratiques MLOps qui permettent d'assurer la traçabilité, la reproductibilité, l'automatisation et la fabrication du cycle de vie des modèles.

Question de recherche :

Comment concevoir et déployer un pipeline MLOps complet permettant de prédire efficacement le risque d'échec scolaire des étudiants tout en garantissant la traçabilité, la reproductibilité et l'industrialisation du modèle de Machine Learning ?

🎯 Objectifs du projet

🎯 Objectif général

L'objectif de ce projet est de développer une solution intelligente de prédiction de la réussite académique des étudiants en s'appuyant sur les techniques de Machine Learning et sur la mise en place d'un pipeline MLOps complet. Cette solution couvre l'ensemble du cycle de vie du modèle, depuis le prétraitement des données et l'entraînement jusqu'au suivi des expérimentations, à la gestion des versions et au déploiement en production, afin de garantir la reproductibilité, la fiabilité et l'industrialisation du système.

🎯 Objectifs spécifiques

Afin d'atteindre l'objectif général, plusieurs objectifs spécifiques ont été définis :

- **EDA & Préparation** : analyse exploratoire, visualisation des corrélations, nettoyage et ingénierie des caractéristiques ;
- **Modélisation** : entraînement et optimisation (Régression Logistique, Arbres de Décision) ;
- **Tracking** : suivi automatique des expériences avec MLflow (*Autolog*) et sélection du modèle champion via *Model Registry* ;
- **Développement Backend** : API REST avec FastAPI exposant le modèle optimal ;
- **Développement Frontend** : interface Web interactive avec Streamlit ;
- **Interconnexion** : communication asynchrone entre Streamlit et FastAPI ;
- **Conteneurisation** : Dockerfile pour l'API Backend ;
- **Déploiement** : API sur Hugging Face Spaces, interface sur Streamlit Community Cloud.

🧪 Méthodologie adoptée

La réalisation de ce projet s'est appuyée sur une démarche structurée inspirée des principes du cycle de vie MLOps [5, 6, 7]. Dans un premier temps,

une analyse exploratoire des données (EDA) a été menée afin de comprendre les caractéristiques du dataset, d'identifier les variables les plus pertinentes et de détecter d'éventuelles anomalies. Les données ont ensuite été préparées à travers différentes opérations de prétraitement, notamment le nettoyage, l'encodage des variables catégorielles et la transformation de la variable cible [3, 4].

Par la suite, plusieurs modèles de Machine Learning ont été entraînés et évalués afin de sélectionner la solution la plus performante pour la prédiction de la réussite académique des étudiants [3, 13]. Les expérimentations, les paramètres et les métriques obtenues ont été suivis à l'aide de MLflow, garantissant ainsi la traçabilité et la reproductibilité des résultats [8].

Le modèle retenu a ensuite été déployé sous la forme d'une API REST développée avec FastAPI [9], puis intégré à une interface utilisateur interactive réalisée avec Streamlit [10]. Enfin, l'application a été conteneurisée à l'aide de Docker [11] et déployée sur le cloud afin d'assurer sa portabilité, sa fiabilité et son accessibilité [12].

Chapitre 1

ÉTAT DE L'ART ET CONCEPTS FONDAMENTAUX

CADRE THÉORIQUE & POSITIONNEMENT

“ L'évolution rapide des technologies numériques et de l'intelligence artificielle a profondément transformé plusieurs domaines, notamment celui de l'éducation. Ce chapitre est dédié à la présentation de l'état de l'art et des concepts théoriques fondamentaux qui soutiennent ce projet. Après avoir défini les principes du Machine Learning et ses applications dans le secteur éducatif, nous détaillerons la transition vers la culture MLOps ainsi que la pile technologique retenue pour l'industrialisation de notre solution. ”

• • • • •

1.1 Intelligence artificielle et Machine Learning

L'intelligence artificielle est actuellement considérée comme un domaine scientifique et technique dont l'objectif et le design de ses systèmes sont capables de simuler les capacités cognitives humaines telles que l'analyse, la réflexion, l'apprentissage et la prise de décision. Elle constitue actuellement un pilier fondamental de la transformation numérique et couvre plusieurs secteurs, les plus importants étant l'éducation, la santé et l'industrie.

1.1.1 Définition de l'intelligence artificielle

L'intelligence artificielle fait référence à un ensemble de méthodes et de techniques qui permettent à la machine informatique de reproduire certaines capacités associées à l'intelligence humaine telles que l'apprentissage, la réflexion, la résolution de problèmes, la prise de décision et l'adaptation à de nouvelles situations. Selon la société IBM, l'intelligence artificielle combine des techniques capables de simuler certaines capacités cognitives humaines en tirant parti des données, des algorithmes et des capacités de calcul avancées. [1].

L'intelligence artificielle constitue aujourd'hui un élément essentiel dans les sciences informatiques modernes et trouve des applications dans de nombreux secteurs différents, comme l'éducation par exemple. Dans le cadre de ce projet, elle est utilisée pour analyser les caractéristiques académiques, comportementales, sociales et économiques des étudiants afin d'identifier les dossiers des étudiants à risque de décrochage scolaire, ainsi que pour aider les établissements éducatifs à mettre en place les mesures préventives nécessaires.

1.1.2 Définition du Machine Learning

L'apprentissage automatique ou apprentissage machine est une branche de l'intelligence artificielle dédiée à la conception de systèmes capables d'apprendre automatiquement à partir des données sans être programmés explicitement pour chaque tâche. Selon des documents officiels de Scilab, les algorithmes d'apprentissage automatique construisent des modèles mathématiques à partir des

données d'apprentissage afin de faire des prévisions ou de prendre des décisions concernant de nouvelles observations.

L'objectif principal de l'apprentissage automatique est d'extraire des connaissances à partir de données historiques et de généraliser ces connaissances à de nouvelles situations inconnues, et cette capacité d'apprentissage permet aux modèles de découvrir les relations complexes entre les variables et de fournir des prévisions éclairées [4].

Dans ce projet, l'apprentissage automatique est utilisé pour construire un modèle prédictif capable d'estimer la probabilité de réussite ou d'échec des étudiants dans leurs études en fonction d'un ensemble de variables décrivant leur environnement familial, leurs habitudes d'étude et leur parcours académique.

1.1.3 Types de Machine Learning et classification

Les approches de Machine Learning sont généralement regroupées en trois catégories principales selon la nature des données disponibles et le mode d'apprentissage adopté par l'algorithme.

Apprentissage supervisé

L'apprentissage supervisé (*Supervised Learning*) repose sur l'utilisation d'un ensemble de données étiquetées où chaque observation est associée à une valeur cible connue. L'objectif de l'algorithme est d'apprendre un modèle qui permet de prédire correctement cette valeur cible à partir des variables d'entrée. Selon la documentation officielle de scikit-learn, cette catégorie comprend en particulier[4].

Dans le cadre de ce projet, la prédiction de la réussite scolaire est formulée comme un problème de classification binaire supervisée où chaque étudiant est associé à l'une des deux classes suivantes : réussite ou échec scolaire.

Apprentissage non supervisé

L'apprentissage non supervisé (*Unsupervised Learning*) est appliqué à un ensemble de données qui ne contient aucune variable cible. L'objectif dans ce cas est de reconnaître automatiquement la structure cachée, les regroupements

ou les relations entre les observations. Parmi les techniques les plus utilisées, il y a les méthodes de regroupement comme K-means, ainsi que les techniques de réduction de dimension. [4].

Ce type d'apprentissage est particulièrement utile pour explorer les données et découvrir des profils similaires au sein d'une population sans avoir besoin d'informations préalablement annotées.

Apprentissage par renforcement

L'apprentissage par renforcement (*Reinforcement Learning*) repose sur l'interaction continue entre un agent intelligent et son environnement. À chaque étape, l'agent effectue une action et reçoit une récompense ou une punition en fonction de la qualité de sa décision, et son objectif est de maximiser la récompense cumulative au fil du temps en apprenant progressivement une stratégie optimale.

Cette approche est largement utilisée dans les domaines de la robotique, des systèmes autonomes, des jeux stratégiques et des systèmes de recommandation intelligents.

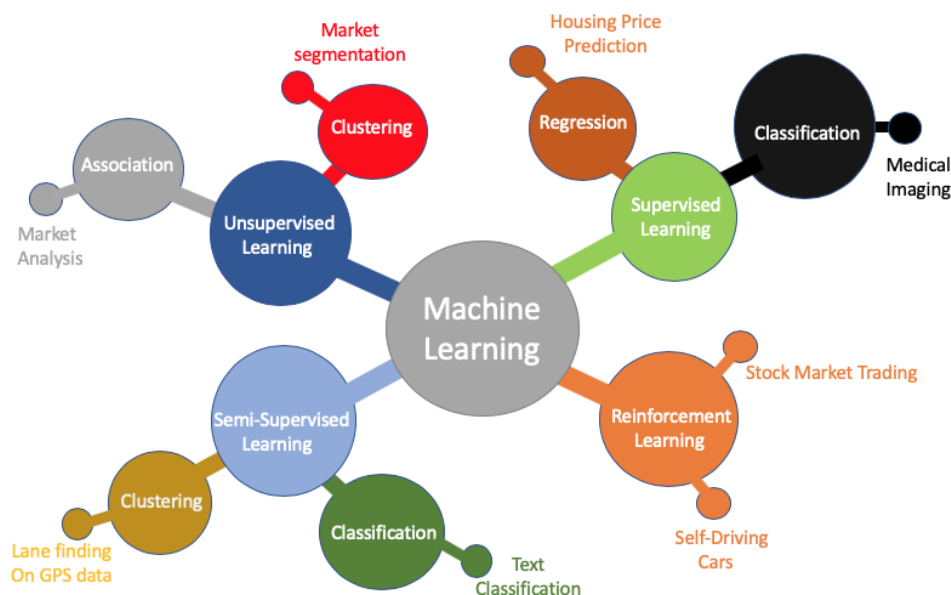


FIGURE 1.1 – Classification des différentes branches du Machine Learning.

1.2 Machine Learning appliqué à l'éducation (EdTech)

Le domaine de l'EdTech (*Educational Technology*) rassemble tous les outils et techniques numériques qui sont mis en œuvre pour améliorer, enrichir et transformer les processus d'apprentissage. L'apprentissage automatique y joue un rôle important en introduisant l'analyse prédictive dans le suivi du processus d'apprentissage, affirment les institutions éducatives.

1.2.1 Analyse des données éducatives

L'établissement éducatif moderne collecte de nombreuses données sur le parcours des apprenants. Ces indicateurs incluent les résultats de chaque étape, le temps quotidien consacré aux études, le contexte socio-économique de la famille, ainsi que différents comportements académiques. L'analyse factorielle de ces variables permet d'isoler les causes fondamentales qui influencent la réussite ou le décrochage.

1.2.2 Détection précoce des étudiants à risque

L'un des principaux objectifs de l'intégration de l'intelligence artificielle dans le milieu académique est la détection précoce des dossiers des étudiants en difficulté ou à risque d'échec scolaire. Identifier ces cas à l'avance permet aux équipes pédagogiques de mettre en œuvre des plans de soutien et des mesures correctives personnalisées avant les examens finaux.

1.3 Vers le MLOps : Limites du ML traditionnel et cycle de vie

Le développement d'un modèle d'apprentissage efficace au sein d'un environnement de recherche ne suffit pas à garantir sa validité et sa valeur scientifique dans la production ; les projets modernes nécessitent un cadre rigoureux pour le génie logiciel afin d'assurer l'exploitation continue des modèles.

1.3.1 Limites des projets ML traditionnels

Les approches de Data Science traditionnelles, souvent isolées au sein de notebooks, révèlent des failles majeures lors de la phase d'industrialisation :

- Absence de suivi systématique des expériences et des versions du modèle. ;
- La difficulté à garantir la possibilité de reproduire les étapes de l'entraînement avec précision ;
- La gestion complexe et artisanale des artefacts de modèles ;
- Absence de pipeline de données automatisé et robuste ;
- Le fossé technique pour le déploiement et la maintenance du modèle en production.

1.3.2 Besoin du MLOps

MLOps est une culture technologique émergente, visant à unifier, sécuriser et automatiser l'ensemble du cycle de vie des modèles [5], afin de réduire le temps de mise sur le marché et de garantir la fiabilité des systèmes d'inférence.

1.3.3 Cycle de vie d'un projet MLOps

Le cycle de vie d'une solution MLOps se conçoit comme une boucle itérative continue structurée autour des étapes suivantes :

- La collecte et l'ingestion des flux de données ;
- Le prétraitement et le nettoyage automatisé des données brutes ;
- L'entraînement, l'ajustement et l'optimisation des hyperparamètres du modèle ;
- L'évaluation multicritère des performances ;
- Le tracking centralisé des expériences ;
- Le déploiement conteneurisé du modèle sélectionné ;
- Le monitoring continu des performances en production[6, 5].

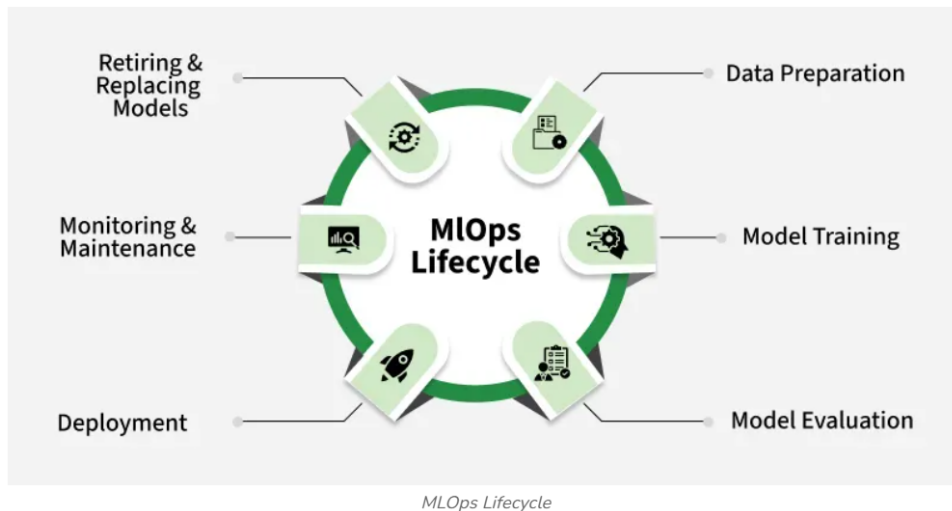


FIGURE 1.2 – Cycle de vie itératif d'un projet de Machine Learning Operations (MLOps).

1.4 Spécifications et choix de la pile technologique

La mise en œuvre de notre architecture logicielle repose sur un écosystème d'outils open source et de plates-formes cloud avancées spécialisées dans la production de modèles avec une efficacité industrielle.

1.4.1 Gestion du cycle de vie des modèles : MLflow

MLflow est une plateforme open source essentielle pour piloter du cycle de vie des algorithmes. Au sein de notre projet, le prend en charge le suivi des expériences en capturant automatiquement les paramètres (*Autologging*) et régit le *Model Registry* afin de certifier et archiver le modèle qualifié pour la production.[8]

1.4.2 Architecture API : FastAPI

FastAPI est un framework web moderne, asynchrone et hautement performant écrit en Python. Il est employé dans ce projet pour encapsuler le modèle mathématique sérialisé et l'exposer sous la forme d'un micro-service REST accessible via des requêtes HTTP sécurisées.[9]

1.4.3 Interface utilisateur : Streamlit

Streamlit est une bibliothèque Python dédiée à la conception rapide d'applications web interactives pour la Data Science. Elle fournit une abstraction complète du développement Frontend, permettant de traduire notre modèle en une interface graphique intuitive pour les utilisateurs finaux.[10]

1.4.4 Isolation de l'environnement : Docker

Docker est une technologie de conteneurisation leader du marché. Elle permet d'isoler notre API FastAPI et l'ensemble de ses dépendances d'exécution au sein d'une image logicielle légère, autonome et immuable (*Container*), éliminant ainsi toute divergence liée aux infrastructures d'accueil.[11].

1.4.5 Plateformes d'hébergement PaaS : Hugging Face Spaces & Streamlit Cloud

La publication sur le cloud garantit la disponibilité et l'accès continu à notre solution en ligne. Pour se conformer aux restrictions de partitionnement dans notre architecture, l'API back-end est déployée de manière isolée dans un espace intégré sur **Hugging Face Spaces**, tandis que l'interface utilisateur est hébergée sur la plateforme Streamlit Community Cloud gérée. **Streamlit Community Cloud**. L'assemblage technique de notre architecture logicielle répond à des exigences strictes de performance, d'interopérabilité et d'agilité. La pile technologique retenue se justifie par les critères d'ingénierie suivants :

- **Python** : Établi comme le standard industriel incontournable pour la manipulation de données, bénéficiant d'un écosystème de bibliothèques mathématiques mature.
- **Scikit-learn** : Choisi pour la robustesse et l'efficacité de ses implémentations des algorithmes de classification (Régression Logistique et Arbres de Décision).
- **MLflow** : Retenu pour sa capacité à centraliser la gouvernance des modèles et à automatiser les phases de comparaison des métriques au sein du

registre.

- **FastAPI** : Préféré aux architectures traditionnelles pour sa rapidité d'exécution, sa gestion native de l'asynchronisme et la génération automatisée de la documentation Swagger.
- **Streamlit** : Sélectionné pour accélérer le prototypage de l'interface graphique sans ajouter de dette technique sur la maintenance d'un framework JavaScript complexe.
- **Docker** : Intégré comme la brique fondamentale de notre stratégie de déploiement continu, garantissant une portabilité totale du Backend entre l'environnement de développement local et l'infrastructure Cloud.

1.5 Conclusion

En somme, ce premier chapitre a permis d'établir les fondations Les théories nécessaires pour comprendre notre approche. Nous avons défini le cadre de travail de l'apprentissage automatique dans les technologies éducatives et met en évidence l'apport des pratiques MLOps pour l'industrialisation de nos modèles. Comprendre ces concepts et vérifier la validité de notre infrastructure technologique ouvrent la voie aux phases de traitement pratique. Le chapitre suivant sera ainsi entièrement consacré à la présentation du jeu de données initial et à la réalisation de l'analyse exploratoire des données (EDA).

Chapitre 2

PRÉPARATION DES DONNÉES ET ANALYSE EXPLORATOIRE

ANALYSE & INGÉNIERIE DES DONNÉES

“ L’ingénierie des données et l’analyse exploratoire constituent des étapes fondamentales dans la construction de tout pipeline MLOps. La qualité et la fiabilité des prédictions d’un modèle de Machine Learning dépendent directement de la pertinence, de la clarté et du traitement appliqué aux variables d’entrée. Ce chapitre est consacré à la présentation du jeu de données éducatives exploité, à l’analyse statistique et graphique de ses composants, ainsi qu’aux différentes étapes de nettoyage, d’encodage, de normalisation, de labellisation binaire de la cible et de partitionnement nécessaires avant la phase de modélisation algorithmique. ”

• • • • •

2.1 Présentation et description générale du dataset

2.1.1 Source des données

Le jeu de données utilisé dans le cadre de ce projet provient de collectes de données scolaires regroupant des informations académiques, sociodémographiques et comportementales sur les apprenants. Ces données brutes ont été initialement partagées et rendues publiques sur le célèbre dépôt de recherche **UCI Machine Learning Repository** [14], issues de travaux de modélisation de la performance des étudiants dans l'enseignement secondaire [13]. Conçu pour anticiper les trajectoires scolaires, ce jeu de données s'inscrit pleinement dans les domaines de l'écosystème EdTech et de l'Educational Data Mining (EDM).

2.1.2 Description générale du dataset

Les données se composent initialement des observations organisées sous forme tabulaire, manipulées à l'intérieur de notre environnement de développement à l'aide de la bibliothèque `Pandas`. Chaque ligne présente un profil unique d'un étudiant, et les colonnes les diverses dimensions de son quotidien : soit des caractéristiques personnelles, environnement familial, habitudes d'étude, indicateurs ou santé, de consommation et de vie sociale. L'objectif principal est d'exploiter ces variables explicatives pour prédire une variable cible représentant la réussite ou le risque d'échec scolaire de l'élève.

2.2 Analyse exploratoire des données (EDA)

L'analyse exploratoire des données (EDA) constitue une étape indispensable pour appréhender la structure interne du dataset, identifier les anomalies exogènes et déceler visuellement les patterns comportementaux [8]. Elle a été menée en exploitant conjointement `Matplotlib` et la bibliothèque `Seaborn` (importée sous l'alias `sns`).

2.2.1 Analyse univariée : Distribution de la variable cible

L'examen de la distribution de la variable cible révèle un comportement général semblable à une distribution gaussienne centrée autour de 10,5/20. Cependant, nous remarquons la présence d'une anomalie statistique importante : un pic distinct d'étudiants ayant obtenu la note stricte de 0. Ce phénomène traduit généralement des situations d'abandon en cours d'année ou d'absentéisme total aux examens plutôt qu'un échec purement académique.

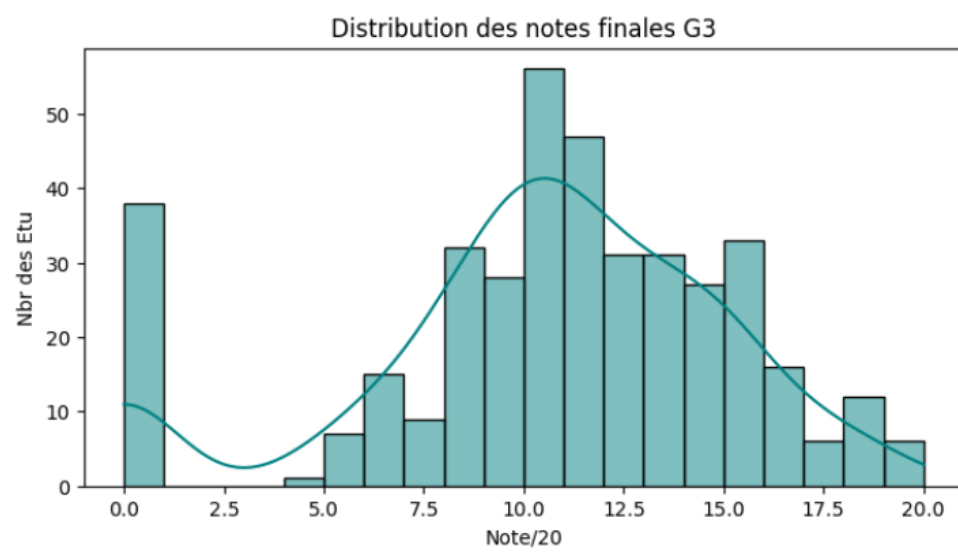


FIGURE 2.1 – Distribution empirique des notes finales G3 et ajustement de densité

2.2.2 Analyse bivariée : Facteurs comportementaux de réussite

L'analyse bivariée met en évidence les facteurs comportementaux liés au succès. L'examen des nuages de points montre deux dynamiques : d'une part, il y a une corrélation positive entre le temps d'étude (`studytime`) et le maintien de bonnes notes., les profils à faible temps de travail ($< 2h$) étant plus exposés aux risques d'échec. D'un autre côté, la projection des absences (`absences`) montre que l'accumulation de journées manquées dégrade significativement la variance des notes, barrant l'accès aux mentions supérieures dès lors qu'un seuil critique est dépassé.

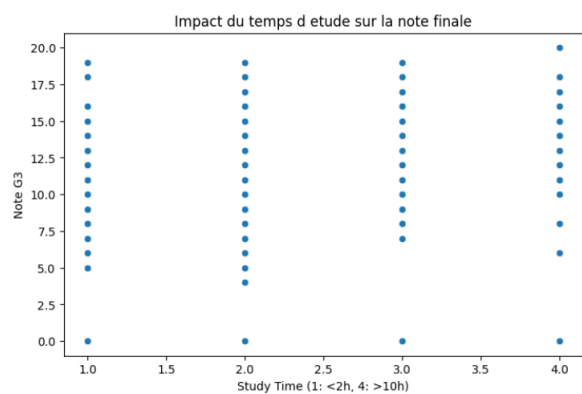


FIGURE 2.2 – Impact du temps d’étude hebdomadaire sur G3

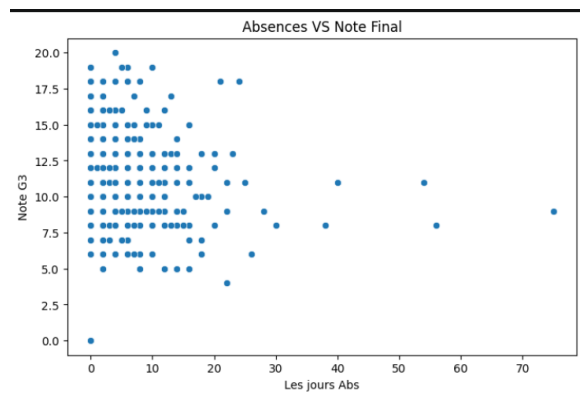


FIGURE 2.3 – Dispersion des notes en fonction des absences

2.2.3 Étude des corrélations initiales

La construction d’une matrice de corrélation linéaire est utilisée pour mesurer l’intensité et la direction des liens mathématiques qui relient nos caractéristiques numériques primaires avant transformation. Cette étape isole l’impact négatif évident des échecs précédents (failures, $r = -0.36$) ainsi que la forte corrélation interne des notes intermédiaires (G1 et G2).

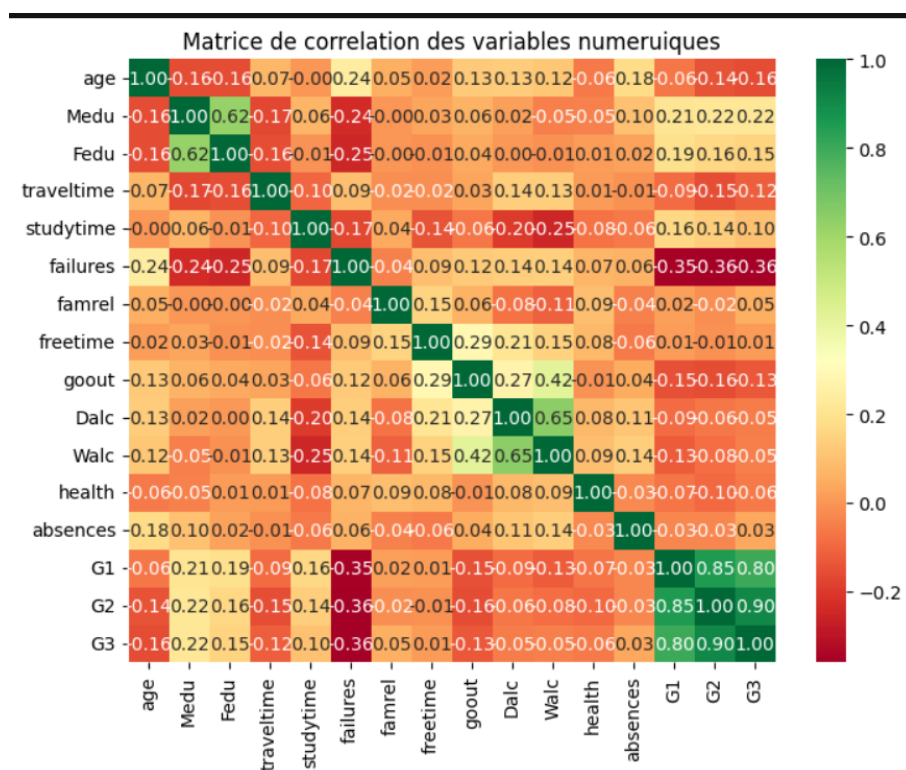


FIGURE 2.4 – Matrice de corrélation de Pearson sur les variables numériques initiales

2.3 Prétraitement des données et feature engineering

Le prétraitement des données (*Data Preprocessing*) formalise notre pipeline de transformation afin de convertir le jeu de données brutes en matrices numériques optimisées et standardisées pour les algorithmes d'apprentissage.

2.3.1 Nettoyage des données et gestion des valeurs manquantes

Cette étape de purification applique un ensemble de filtres stricts qui visent à garantir la fiabilité de la source d'information. Elle comprend la détection et la suppression systématique des lignes dupliquées, la correction des incohérences dans la saisie du texte et la vérification des types de données. Afin de protéger notre pipeline de production contre les ruptures de flux lors des phases d'entraînement ou d'inférence en temps réel, une vérification des valeurs manquantes a été opérée; L'ensemble de données original qui avait une structure complète, n'avait pas besoin de compensation par la moyenne ou le mode, ce qui a permis de conserver l'échantillon tel quel.

2.3.2 Encodage des variables catégorielles

Les modèles de classification de la bibliothèque `Scikit-learn` nécessitent seulement des structures de données numériques. Les attributs textuels qualitatifs du dataset ont donc fait l'objet d'une vectorisation stricte via deux modules importés :

- **OrdinalEncoder / LabelEncoder** : utilisé pour convertir directement les variables catégoriques binaires ou ordinales (comme `sex`, `higher`, `internet`) en indicateurs numériques discrets.
- **OneHotEncoder** : Il est utilisé pour encoder complètement les variables nominales à plusieurs catégories, créant ainsi des colonnes binaires denses 0.1 pour chaque motif afin d'éviter d'introduire une hiérarchie artificielle erronée.

2.3.3 Sélection finale des caractéristiques

Après toutes les opérations de nettoyage et le codage des variables catégoriques, l'étape de sélection des caractéristiques a été effectuée pour construire un jeu de données approprié à la modélisation. L'objectif était de ne conserver que les variables ayant une importance directe dans la prédiction du succès académique.

De plus, les variables G1 et G2, correspondant aux notes obtenues lors des évaluations intermédiaires, ont été délibérément exclues de l'analyse. En effet, ces variables sont fortement liées à la note finale G3 et leur utilisation entraînerait en réalité un risque de fuite d'informations Data Leakage, ce qui pourrait conduire à une performance artificiellement élevée du modèle.

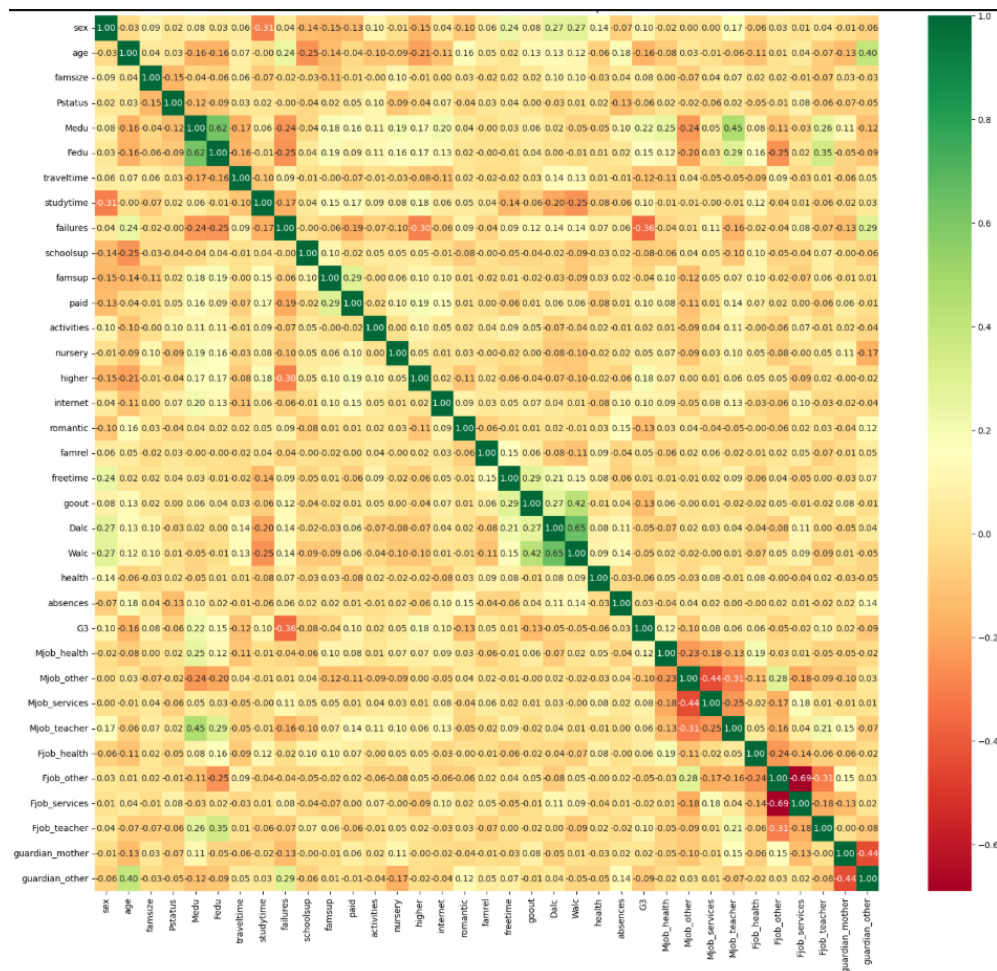


FIGURE 2.5 – Matrice de corrélation globale après encodage des variables catégorielles

Le sous-ensemble final comprend 15 variables explicatives sélectionnées

après la préparation et le nettoyage des données. La liste complète de ces variables est présentée dans le tableau résumé ci-dessous 2.1 qui structure la liste `select_features` requise par notre modèle et validée par notre future API. Ces caractéristiques représentent différents facteurs académiques, familiaux et sociaux pouvant influencer la réussite des étudiants.

TABLE 2.1 –  DICTIONNAIRE DES CARACTÉRISTIQUES — VERSION FINALE

Variable	Type	Description détaillée
age	Numérique	Âge de l'étudiant (en années).
sex	Catégoriel	Sexe de l'étudiant : 'F' (Féminin), 'M' (Masculin).
Medu	Numérique	Niveau d'éducation de la mère (échelle de 0 à 4).
Fedu	Numérique	Niveau d'éducation du père (échelle de 0 à 4).
studytime	Numérique	Temps d'étude hebdomadaire : 1 (<2h), 2 (2-5h), 3 (5-10h), 4 (>10h).
failures	Numérique	Nombre d'échecs antérieurs (compris entre 0 et 3, plafonné).
absences	Numérique	Nombre total d'absences cumulées (échelle de 0 à 93).
goout	Numérique	Fréquence des sorties entre amis : de 1 (très faible) à 5 (très élevée).
Walc	Numérique	Consommation d'alcool le week-end : de 1 (très faible) à 5 (très élevée).
health	Numérique	État de santé actuel : de 1 (très mauvais) à 5 (très bon).
higher	Catégoriel	Souhait de poursuivre des études supérieures : 'yes' ou 'no'.
internet	Catégoriel	Accès à Internet au domicile : 'yes' ou 'no'.
traveltime	Numérique	Temps de trajet domicile-école : 1 (<15 min), 2 (15-30 min), 3 (30-60 min), 4 (>1h).
freetime	Numérique	Temps libre après l'école : de 1 (très faible) à 5 (très élevé).
famrel	Numérique	Qualité des relations familiales : de 1 (très mauvaise) à 5 (excellente).

 Ensemble final de 15 caractéristiques clés extraites du jeu de données `student-mat.csv`.

2.4 Labellisation et partitionnement du dataset

2.4.1 Transformation et labellisation de la variable cible

Dans sa structure originale, le niveau de réussite académique finale de l'étudiant est enregistré sous forme de note quantitative continue via la variable `G3`. Pour répondre à notre problème qui se concentre sur la prédiction et la reconnaissance précoce des fichiers de classification binaire supervisés.

Une labellisation stricte a été implémentée en introduisant une variable de seuil réglementaire fixée selon les standards académiques nationaux à 10/20 (`seuil = 10`). La logique de transformation applique un filtre logique suivi

d'un transtypage de type entier (`astype('int')`) permettant de générer les deux classes cibles distinctes :

- **Classe 1 (Réussite scolaire) :** Attribuée lorsque la note finale G3 est supérieure ou égale à 10.
- **Classe 0 (Échec scolaire) :** Attribuée lorsque la note finale G3 est strictement inférieure à 10.

Le bloc de code Python matérialisant cette transformation s'énonce ainsi :

```
y_train_class = (y_train['G3'] >= seuil).astype('int')
y_test_class   = (y_test['G3']   >= seuil).astype('int')
```

Cette étape clé structure mathématiquement la variable cible avant son injection dans le pipeline d'apprentissage.

2.4.2 Séparation des données d'entraînement et de test

Pour s'assurer d'une évaluation juste, éviter les phénomènes de surapprentissage et vérifier de manière objective la possibilité de généraliser la solution, le jeu de données nettoyé et transformé est divisé en utilisant la fonction `train_test_split` de la bibliothèque `Scikit-Learn`. Les données sont divisées selon la répartition standard suivante :

- **L'ensemble d'entraînement (*Train Set*) - 80% :** Dédié exclusivement à l'apprentissage des algorithmes, permettant d'ajuster les poids internes et les frontières de décision des classifieurs.[4]
- **L'ensemble de test (*Test Set*) - 20% :** Préservé de manière totalement isolée, il sert de jeu de validation indépendant pour mesurer les performances et les métriques finales sur des profils étudiants inédits.[4]

2.5 Conclusion

Ce deuxième chapitre m'a permis de préparer correctement les données avant de passer à la modélisation. J'ai nettoyé le dataset, encodé les variables catégorielles et transformé la note G3 en une variable binaire selon le seuil de 10/20.

Bien que complexes, ces étapes de traitement et d'encodage étaient indispensables pour garantir la convergence et la fiabilité des futurs modèles. Le chapitre suivant sera consacré à l'entraînement des modèles et au suivi des expériences avec MLflow.

Chapitre 3

MODÉLISATION MACHINE LEARNING ET GESTION DU CYCLE DE VIE DES MODÈLES

MODÉLISATION SUIVI MLOPS

“ La phase de modélisation algorithmique et d’évaluation structure la capacité prédictive tandis que l’automatisation du suivi et du déploiement en garantit la viabilité opérationnelle. Dans le cadre d’une approche MLOps (Machine Learning Operations), le développement de modèles prédictifs ne peut s’affranchir d’une traçabilité totale et d’un contrôle strict de leur cycle de vie. Ce chapitre détaille la méthodologie statistique adoptée pour l’apprentissage, présente les architectures de classification employées, expose les résultats empiriques réels de nos modèles, et formalise l’infrastructure de tracking et de gestion du registre centralisé construite autour de la plateforme MLflow. ”

3.0.1 Automatisation du pipeline et reproductibilité des données

L'application de notre processus de modélisation repose sur le système open source `Scikit-Learn`. Cette étape permet d'ajuster les fonctions mathématiques de décision sur un ensemble de données d'apprentissage afin de prédire les trajectoires de réussite ou d'échec des étudiants en fonction des caractéristiques comportementales et démographiques identifiées lors de la phase d'ingénierie des données.

La vérification des performances repose sur une séparation stricte entre l'entraînement et le test (Train/Test), assurée en fixant une valeur aléatoire constante (`random_state = 42`), ce paramètre dans le code garantit une reproductibilité totale des partitions des matrices d'entrée. Dans le cadre de notre architecture MLOps, ce découpage n'est plus une action manuelle, mais un composant logiciel automatisé au sein de notre pipeline d'industrialisation, permettant de simuler fidèlement le comportement du système face à des flux de données inédits lors de son exposition future en production.

3.1 Présentation et entraînement des modèles de classification

Deux familles d'algorithmes complémentaires ont été instanciées au sein du pipeline afin de confronter une approche mathématique linéaire à une approche logique non paramétrique.

3.1.1 Stratégie d'entraînement et de partitionnement

L'ajustement des paramètres internes de nos classificateurs suit un modèle d'apprentissage supervisé. L'ensemble de données global, après nettoyage et transformation en vecteurs, a été divisé de manière spécifique en deux sous-ensembles distincts :

- **L'ensemble d'entraînement (*Train Set*)** : Représentant 80% des observations, il est injecté pour guider la mise à jour des poids des modèles lors de la phase d'ajustement (`fit`).

- **L'ensemble de test (*Test Set*)** : Représentant 20% des données résiduelles, il est préservé de toute contamination et sert exclusivement d'échantillon de validation externe pour quantifier la capacité de généralisation de la solution.

3.1.2 Régression logistique

La régression logistique est un classificateur linéaire qui modélise la probabilité d'appartenir à la classe cible en appliquant la fonction sigmoïde à une combinaison linéaire des variables indépendantes :

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)}} \quad (3.1)$$

Dans notre script, ce modèle intègre l'hyperparamètre `class_weight='balanced'` afin d'ajuster dynamiquement les pénalités de la fonction de coût de manière inversement proportionnelle aux fréquences des classes, compensant ainsi le déséquilibre structurel du dataset.

3.1.3 Arbre de décision

L'arbre de décision est un algorithme non paramétrique qui divise de manière itérative l'espace des données en sous-rectangles distincts. À chaque nœud, l'algorithme choisit la variable explicative et le seuil qui améliorent la pureté des sous-groupes, mesurée ici par l'indice d'impureté de Gini. Ce modèle a été retenu pour sa forte interprétabilité métier, matérialisée par des règles de décision explicites de type *If-Then*, hautement valorisables par les équipes pédagogiques.

3.2 Métriques d'évaluation

L'évaluation multicritère de la qualité des prédictions s'articule autour d'indicateurs cardinaux de la classification binaire :

- **L'Exactitude (*Accuracy*)** : Ratio entre le nombre de prédictions correctes et le total des instances évaluées :

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN} \quad (3.2)$$

- **La Précision (*Precision*)** : Proportion d'étudiants réellement en réussite parmi l'ensemble des profils prédits positivement par le modèle. Elle quantifie la fiabilité du diagnostic positif :

$$\text{Précision} = \frac{VP}{VP + FP} \quad (3.3)$$

- **Le Rappel (*Recall* / *Sensibilité*)** : Capacité du système à capturer l'intégralité des individus appartenant à la classe d'intérêt (notamment sa sensibilité à identifier les étudiants en situation d'échec ou de décrochage précoce) :

$$\text{Rappel} = \frac{VP}{VP + FN} \quad (3.4)$$

- **Le Score F1 (*F1-score*)** : Moyenne harmonique pondérée de la précision et du rappel, fournissant un indicateur unique et robuste pour arbitrer l'équilibre des modèles sur des populations asymétriques :

$$\text{F1-score} = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (3.5)$$

- **La Matrice de Confusion** : Tableau croisé de contingence bidimensionnel confrontant les vérités du terrain (*Ground Truth*) aux classes prédites. Elle permet d'isoler analytiquement les Vrais Positifs (VP), Vrais Négatifs (VN), Faux Positifs (FP, erreurs de type I) et Faux Négatifs (FN, erreurs de type II).

3.3 Résultats et synthèse comparative des modèles

Les modèles ont été entraînés de manière iso-périmétrique sur les données d'apprentissage brutes afin de comparer prédictivement leurs pouvoirs discriminants respectifs lors de la phase d'inférence sur le jeu de test indépendant.

3.3.1 Résultats et métriques de la Régression Logistique (Modèle Champion)

Le modèle de régression logistique offre des performances particulièrement solides, soutenu par la régularisation des poids des classes :

- **Accuracy** : 75,95% (0.75949)
- **F1-Score** : 81,55% (0.81553)

La matrice de confusion générée expérimentalement sur le jeu de test s'exprime comme suit :

$$M_{\text{confusion (LREG)}} = \begin{pmatrix} 18 & 9 \\ 10 & 42 \end{pmatrix} \quad (3.6)$$

L'analyse montre que sur un total de 27 étudiants en situation d'échec réel (classe 0), le modèle en identifie correctement 18 (Vrais Négatifs) et commet 9 erreurs de type I (Faux Positifs). Concernant la classe de réussite (classe 1), 42 profils sont correctement qualifiés (Vrais Positifs) face à 10 omissions (Faux Négatifs).

Afin de projeter la supériorité prédictive de ce modèle champion retenu pour la production, ses courbes métriques d'apprentissage ont été générées. La figure 3.1 détaille sa répartition graphique des classes, tandis que la figure 3.4 valide la stabilité opérationnelle du moteur via l'analyse des sensibilités ROC et Précision-Rappel.

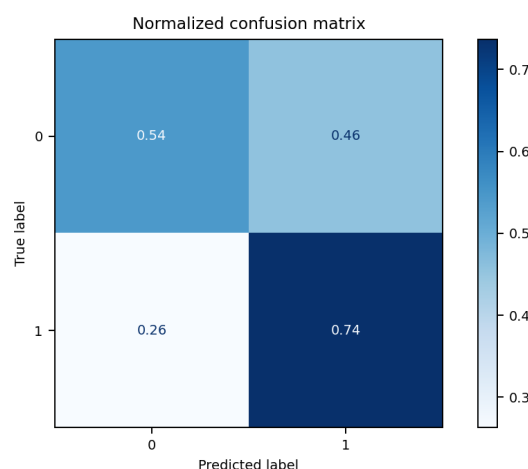


FIGURE 3.1 – Matrice de confusion graphique obtenue sur l'ensemble de validation pour le modèle champion

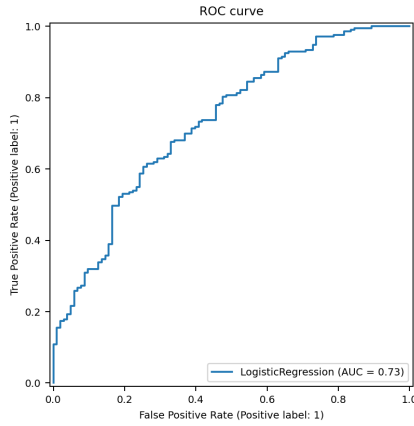


FIGURE 3.2 – Courbe ROC et mesure de l’AUC

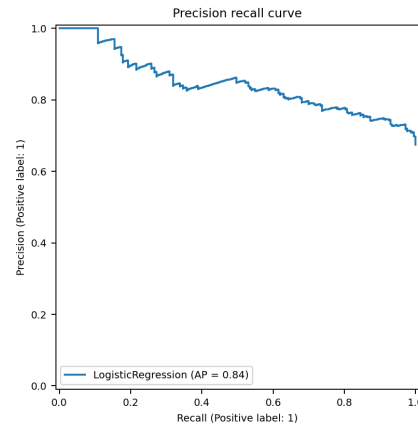


FIGURE 3.3 – Courbe Précision-Rappel

FIGURE 3.4 – Courbes de performance et analyse de sensibilité du classifieur champion

3.3.2 Résultats et métriques de l’Arbre de Décision

L’arbre de décision initial, limité par une profondeur maximale fixe `max_depth=5`, montre une performance inférieure au modèle linéaire :

- **Accuracy** : 70,89% (0.70886)
- **F1-Score** : 79,28% (0.79279)

Sa matrice de confusion associée affiche la répartition suivante :

$$M_{\text{confusion (Arbre)}} = \begin{pmatrix} 12 & 15 \\ 8 & 44 \end{pmatrix} \quad (3.7)$$

Ce modèle commet 15 erreurs de type I, laissant s’échapper un volume critique d’étudiants en échec réel sans détection précoce, ce qui limite son utilité métier immédiate sans optimisation fine de ses hyperparamètres.

3.3.3 Synthèse comparative des performances

La régression logistique prouve qu’elle est plus stable et efficace sur l’échantillon de test global, montrant une augmentation de la précision de +5,06% et un meilleur équilibre de la sensibilité sur la sous-catégorie des échecs par rapport à l’arbre de décision de base. Cette haute capacité de discrimination est ce qui motive sa promotion finale.

3.4 Tracking des expériences avec MLflow

Le passage d'un script de modélisation artisanal à un pipeline de production nécessite l'intégration d'un serveur de tracking capable d'historiser l'ensemble des cycles d'expérimentations.

3.4.1 Présentation de MLflow

MLflow est une suite logicielle open-source dédiée au cycle de vie du Machine Learning (*ML lifecycle management*). Son composant *MLflow Tracking* fournit une API et une interface utilisateur (UI) centralisée pour journaliser de manière immuable les paramètres d'exécution, les versions de code, les métriques d'évaluation et les artefacts de modèles.

3.4.2 Utilisation de MLflow Autolog

Afin d'automatiser la capture des métadonnées d'apprentissage sans alourdir le code source, la journalisation automatique a été initialisée via la commande :

```
mlflow.sklearn.autolog()
```

Lors de l'appel de la méthode `.fit()`, ce composant intercepte le workflow Scikit-Learn pour enregistrer nativement l'arbre aussi LREG complet des dépendances logicielles (*lineage information*), le dictionnaire des hyperparamètres, ainsi que l'empreinte de sérialisation.

3.4.3 Suivi des métriques et paramètres

Une expérience dédiée a été instanciée dans l'interface MLflow sous le nom officiel : "Prédiction de la réussite des étudiants Version Final De tracking". Deux processus itératifs de recherche d'hyperparamètres ont été exécutés et suivis :

1. **Optimisation de l'Arbre de Décision** : Une boucle exploratoire analysant les profondeurs `max_depth ∈ {3, 4, 6, 7}` a généré des runs distincts (ex : `DecisionTree_depth_3`), traçant l'évolution de l'accuracy et du score F1 à chaque étape.

2. **Analyse de convergence de la Régression Logistique** : L'évaluation de la tolérance du solveur à travers les valeurs $\text{max_iter} \in \{100, 500, 1000\}$ a été consignée sur le serveur.

Le tableau de bord global matérialisant le suivi et le comparatif de ces exécutions au sein de notre serveur local est présenté dans la figure 3.5.

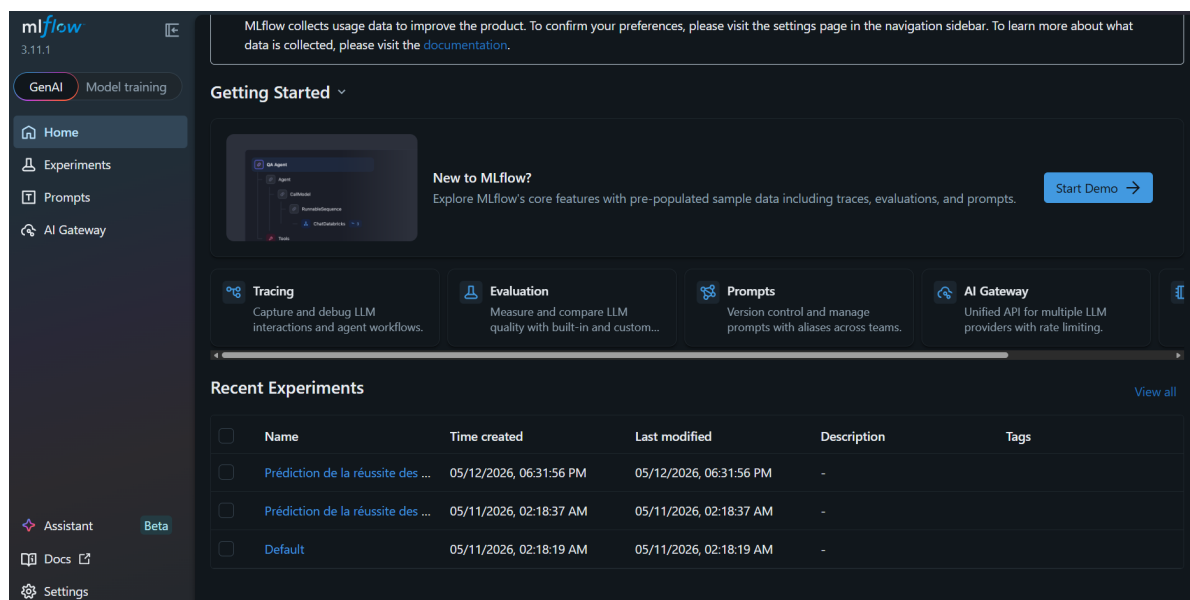


FIGURE 3.5 – Tableau de bord de l'interface locale MLflow UI et historique complet des runs

L'inspection détaillée de l'expérience, telle qu'illustrée par la figure 3.6, permet de valider la structuration unifiée des runs associés aux différents algorithmes testés.

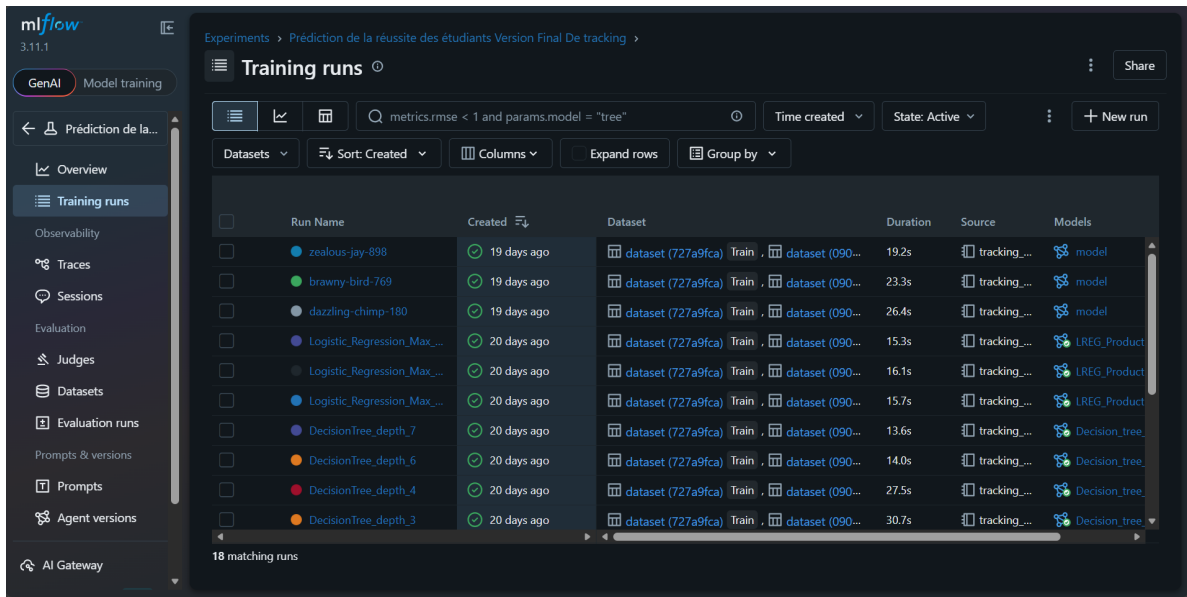


FIGURE 3.6 – Détails des exécutions et métadonnées associées au projet de tracking sur l’interface MLflow

3.4.4 Comparaison des expériences et alertes de non-convergence

L’interface graphique de MLflow a permis d’analyser l’historique des exécutions. Un comportement d’ingénierie critique a été intercepté lors du run configuré avec `max_iter=100`. Le système a levé une alerte formelle de non-convergence :

```
ConvergenceWarning: lbfgs failed to converge after
100 iteration(s)
```

Cette métadonnée, capturée par le serveur de tracking, démontre empiriquement que la limite de 100 itérations est insuffisante pour stabiliser l’optimiseur numérique sur ce jeu de données, justifiant la sélection opérationnelle des runs à 1000 itérations où le gradient se stabilise de manière optimale.

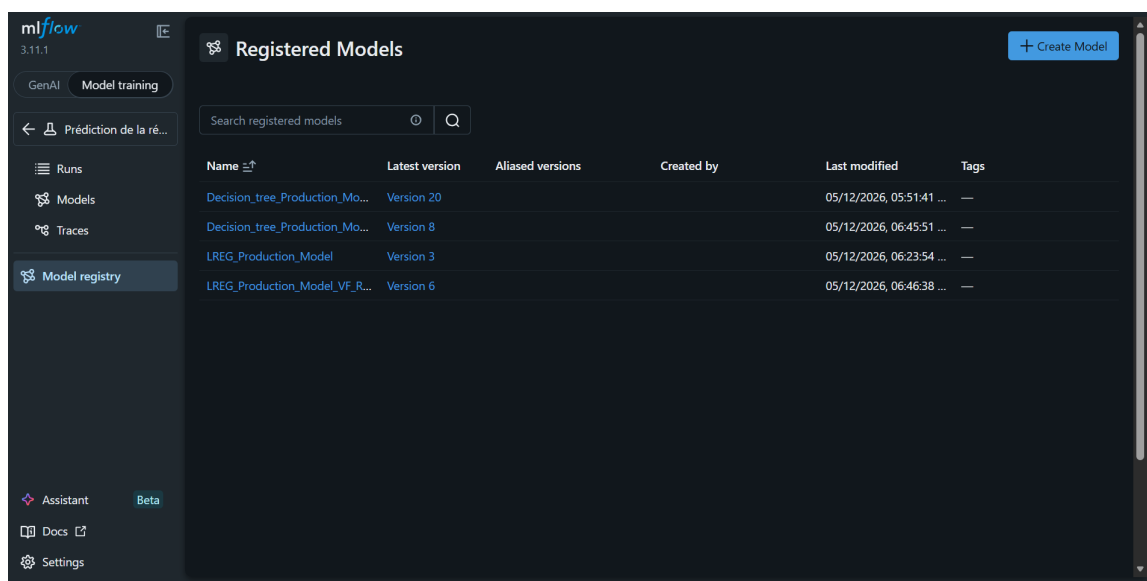
3.5 Gestion du Model Registry

Le composant *Model Registry* de MLflow fournit un magasin centralisé et des API permettant de gérer l’évolution, le versionnage et le statut de déploiement des modèles de production.

3.5.1 Enregistrement des modèles

À la fin de chaque exécution managée, les modèles validés sont poussés vers le registre centralisé via l'instruction `mlflow.sklearn.log_model()`. Nos processus ont alimenté deux registres nommés uniques (Figure 3.7) :

- "Decision_tree_Production_Model_VF_DT" (générant successivement les versions logicielles d'artefacts 5, 6, 7 et 8).
- "LREG_Production_Model_VF_REG" (contenant les versions successives du modèle linéaire).



The screenshot shows the MLflow web interface with the 'Registered Models' tab selected. A table lists the registered models with columns for Name, Latest version, Aliased versions, Created by, Last modified, and Tags. The table contains four entries: 'Decision_tree_Production_Model_VF_DT' with versions 20 and 8, and 'LREG_Production_Model_VF_REG' with versions 3 and 6. The interface also includes a search bar, a 'Create Model' button, and a sidebar with navigation options like 'Runs', 'Models', 'Traces', and 'Model registry'.

Name	Latest version	Aliased versions	Created by	Last modified	Tags
Decision_tree_Production_Model_VF_DT	Version 20			05/12/2026, 05:51:41 ...	—
Decision_tree_Production_Model_VF_DT	Version 8			05/12/2026, 06:45:51 ...	—
LREG_Production_Model_VF_REG	Version 3			05/12/2026, 06:23:54 ...	—
LREG_Production_Model_VF_REG	Version 6			05/12/2026, 06:46:38 ...	—

FIGURE 3.7 – Vue globale des modèles enregistrés au sein du registre centralisé MLflow

3.5.2 Sélection et promotion du modèle champion

Pour automatiser le déploiement continu sans intervention humaine, une fonction d'arbitrage logicielle appelée `promote_the_best_model_to_Production` a été développée. Cette routine interroge via l'API de `MLflowClient` pour passer en revue toutes les opérations liées à l'expérience. Elle filtre et isole l'identifiant unique (`best_run_id`) qui a obtenu la valeur maximale pour la métrique de précision (Accuracy) sur le jeu de test, déterminant ainsi de manière objective le modèle gagnant pour la régression logistique.

Une fois le modèle champion extrait, le script orchestre sa transition de phase logicielle via l'API de gouvernance de MLflow pour le promouvoir officielle-

ment à l'état "Production". Comme le met en évidence la figure 3.8, la version stable 4 de notre régression logistique (LREG_Production_Model_VF_REG) porte le label actif Production, la rendant disponible pour requêtage par des applications tierces.

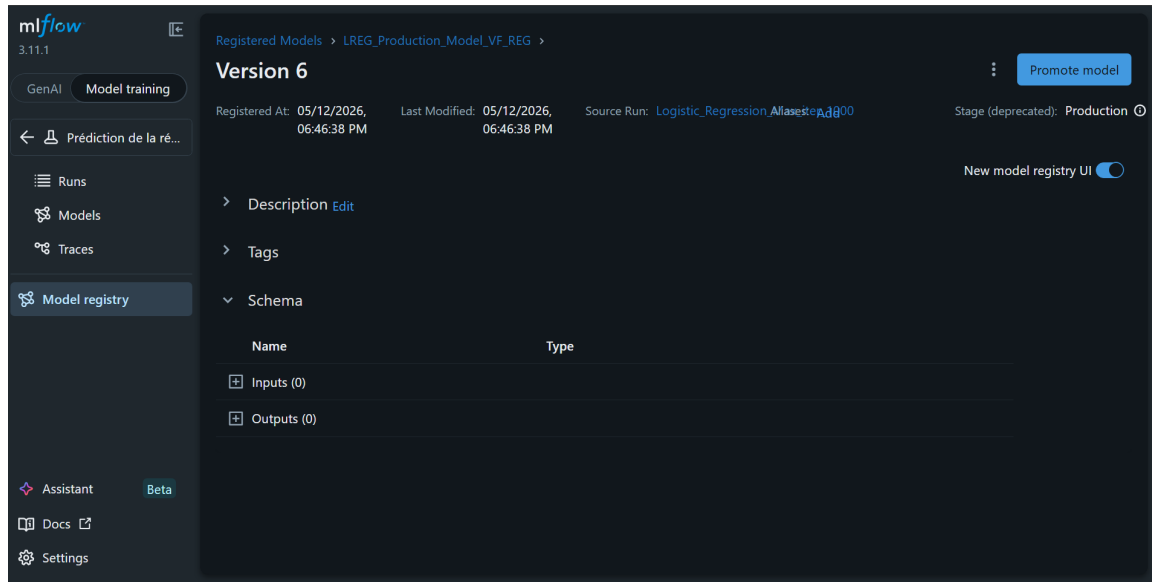


FIGURE 3.8 – Statut du registre pour la Régression Logistique affichant la version championne promue en Production

Cette commande publie immédiatement la version sélectionnée, en passant automatiquement les anciennes versions actives à l'état "Archived", éliminant ainsi tout risque de conflit entre les versions.

3.6 Conclusion

Dans ce chapitre, j'ai entraîné et comparé deux modèles : la Régression Logistique et l'Arbre de Décision. Les résultats ont montré que la Régression Logistique donnait de meilleures performances avec une accuracy de 75,95% et un F1-score de 81,55%. Ce qui m'a surpris c'est que l'Arbre de Décision, malgré sa complexité visuelle, était moins performant. MLflow m'a beaucoup aidé à comparer les résultats des différentes expériences sans me perdre dans mes notes. Le modèle final a été promu en Production dans le Model Registry, prêt à être utilisé par l'API.

Chapitre 4

INDUSTRIALISATION ET DÉPLOIEMENT DE LA SOLUTION

INDUSTRIALISATION & DÉPLOIEMENT MLOPS

“ Le passage d’un prototype de recherche à une application prête pour la production constitue le cœur de la démarche MLOps (Machine Learning Operations). L’industrialisation vise à encapsuler le modèle prédictif sélectionné au sein d’une infrastructure logicielle standardisée pour le rendre accessible, résilient, portable et facilement maintenable. Ce chapitre est consacré à la présentation de l’architecture globale du système, au développement de l’interface de programmation d’application (API) avec FastAPI, à sa conteneurisation via Docker, ainsi qu’au déploiement de sa couche backend sur Hugging Face Spaces. Enfin, nous détaillerons la conception de la couche utilisateur développée sous Streamlit, son hébergement sur Streamlit Community Cloud, et les protocoles de tests de bout en bout validant le fonctionnement de la solution. ”

4.1 Architecture logicielle globale de la solution

Pour assurer le maximum de scalabilité et de compatibilité avec les standards modernes d'ingénierie logicielle, la solution technique repose sur une architecture découplée de type microservices. Cette architecture organise une séparation claire des responsabilités entre le traitement des requêtes, le stockage des éléments et l'affichage de l'interface utilisateur.

L'architecture globale s'articule autour de six blocs fonctionnels interconnectés :

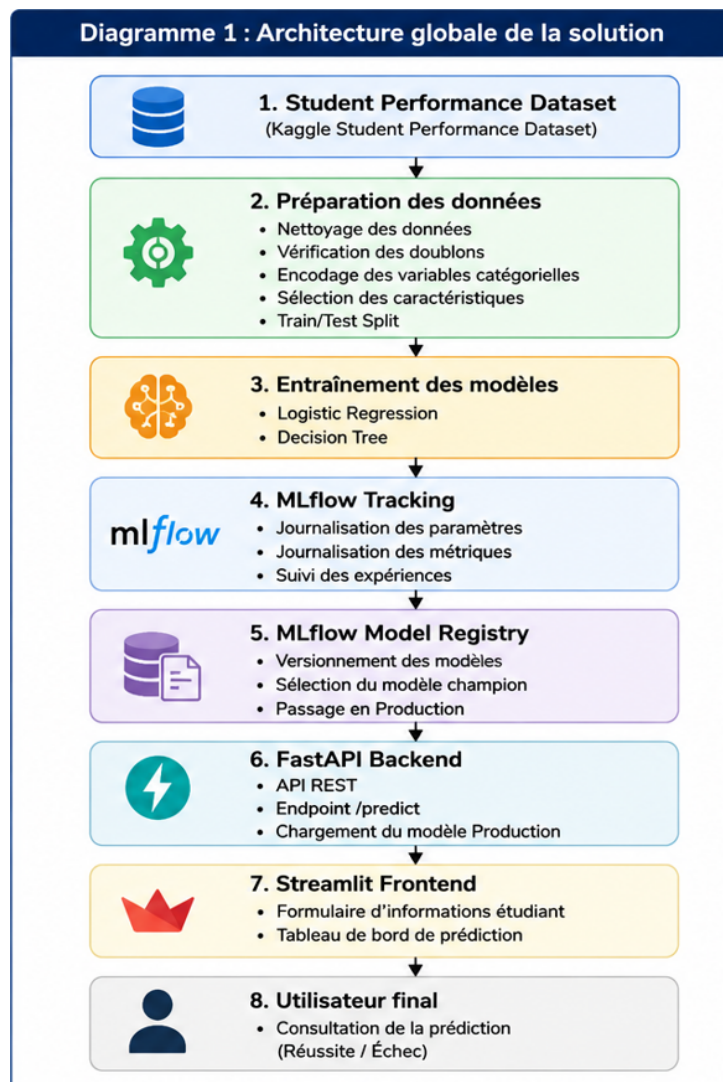


FIGURE 4.1 – Architecture globale du système MLOps pour la prédiction des résultats scolaires

La diffusion continue de l'arbre du code source et de ses dépendances constitue la base de cette architecture. À cet effet, l'intégralité du projet a été centralisée et structurée sur un dépôt distant sécurisé de la plateforme GitHub (Figure 4.2), facilitant la traçabilité des modifications logicielles.

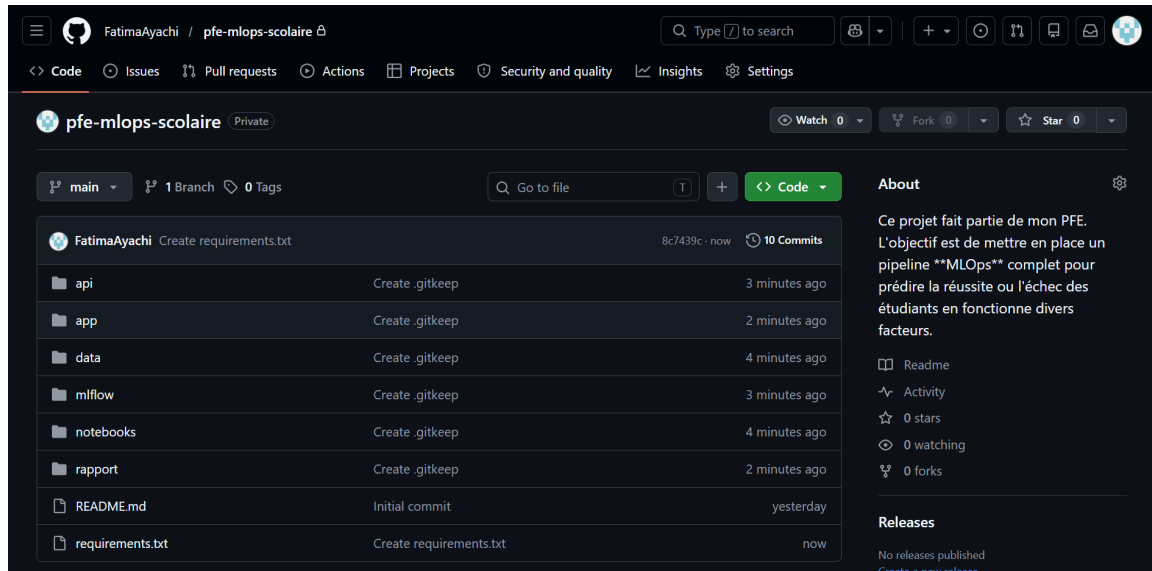


FIGURE 4.2 – Structure de l'arborescence du projet et versioning sur le dépôt distant GitHub

4.2 Développement du Backend avec FastAPI

Le moteur de service backend a été développé en utilisant le framework FastAPI. Ce choix technologique est motivé par ses performances élevées, sa gestion native de la programmation asynchrone (`async/await`), et sa génération automatique de documentations interactives au standard OpenAPI.

4.2.1 Structure du projet

Afin de structurer le code de manière modulaire et d'éviter les fichiers monolithiques, le répertoire backend est organisé selon l'arborescence standard suivante :

```
api/
  main.py                # Point d'entrée de l'application Fast
  Dockerfile             # Instructions de conteneurisation
  requirements.txt        # Liste des dépendances logicielles str
```

```
model/
    champion_model.pkl # Artefact sérialisé du modèle de pr
```

4.2.2 Création des endpoints et validation des schémas

Le point d'entrée de l'application annonce les chemins réseau pour l'API. Le point central du système est le chemin de prédiction, accessible via la méthode HTTP POST à l'adresse `/predict`. Pour sécuriser les flux d'entrée, la validation des types est déléguée à la bibliothèque `Pydantic` à travers un schéma de données héritant de la classe `BaseModel`.

Chargement du modèle et génération des prédictions

Pour réduire le temps de latence lors de l'inférence et éviter les lectures répétées du disque à chaque requête, le fichier de modèle (`model.pkl`) est chargé en permanence dans la mémoire RAM de manière globale lors du démarrage de l'application FastApi via le module `joblib`. La figure 4.2 présente la structure globale de l'API, incluant la définition du schéma des données d'entrée et le point de terminaison pour les requêtes.

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "model.pkl")
model = joblib.load(MODEL_PATH)
print("Le model a ete charger ")
```

(a) Chargement global du modèle

```
@app.post("/predict")
async def predection_endpoint(data: Student_input):
    data_input = data.dict()
    data_frame = pd.DataFrame([data_input])
    prediction = model.predict(data_frame)
    return {"The result is " : int(prediction[0])}
```

(b) Point de terminaison `/predict`

FIGURE 4.3 – Structure de l'API FastAPI : Logique de chargement du modèle (gauche) et fonction d'inférence (droite)

Lorsqu'une requête POST est envoyée sur le point de terminaison `/predict`, les données reçues sous forme de dictionnaire sont converties en un `DataFrame` de la bibliothèque `pandas`. Ce dernier est injecté dans le modèle pour générer la prédiction en temps réel. L'API encapsule ensuite cette réponse dans un dictionnaire JSON structuré renvoyé au client sous la forme suivante :

```
{  
  "The result is ": 1  
}
```

4.3 Tests fonctionnels de l'API et validation des contrats

Avant toute phase de déploiement dans le cloud, la vérification fonctionnelle et comportementale des flux a été testée localement. Ce protocole expérimental a été initialement structuré autour de l'outil d'ingénierie des réseaux Postman, ce qui permet de simuler l'envoi des requêtes des clients dans des conditions réelles.

Comme l'illustre la figure 4.4, les tests effectués ont permis de valider avec rigueur la bonne réception et le routage des flux, confirmés par le code de statut HTTP 200 OK, ainsi que la conformité syntaxique de la charge utile JSON retournée par le serveur backend.

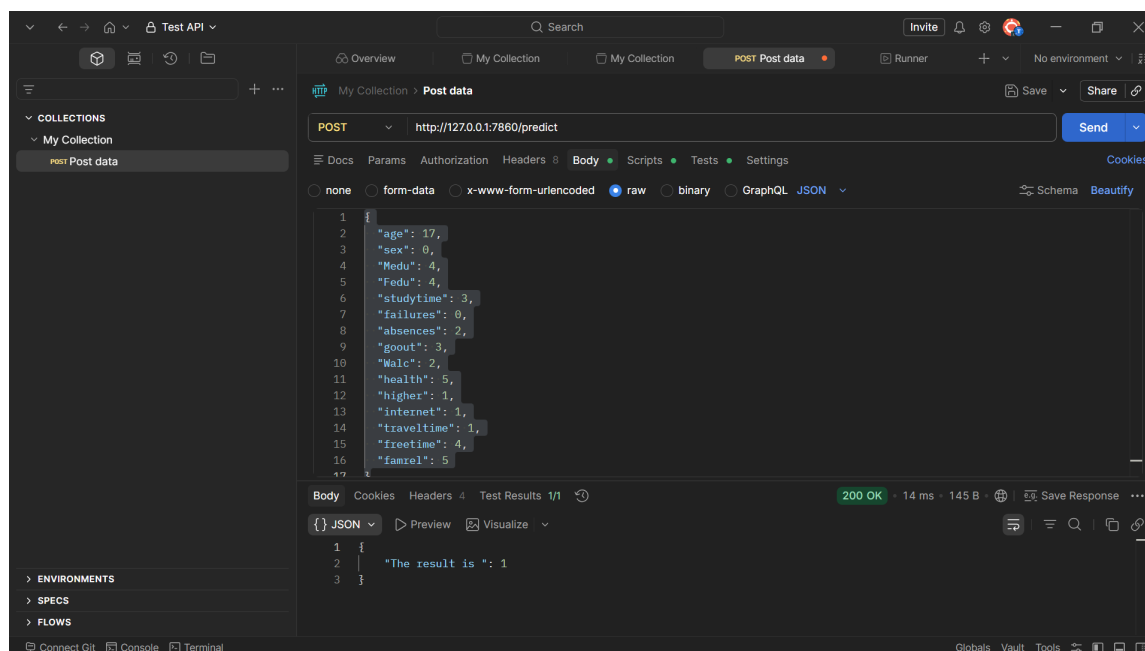


FIGURE 4.4 – Validation fonctionnelle de l'endpoint de prédiction et analyse de la réponse via Postman

En complément, l'interface interactive intégrée Swagger UI au standard OpenAPI a été exploitée. Comme mis en évidence sur la figure 4.5, cet outil liste de

manière exhaustive l'ensemble des routes HTTP exposées ainsi que la structure attendue des schémas de données.

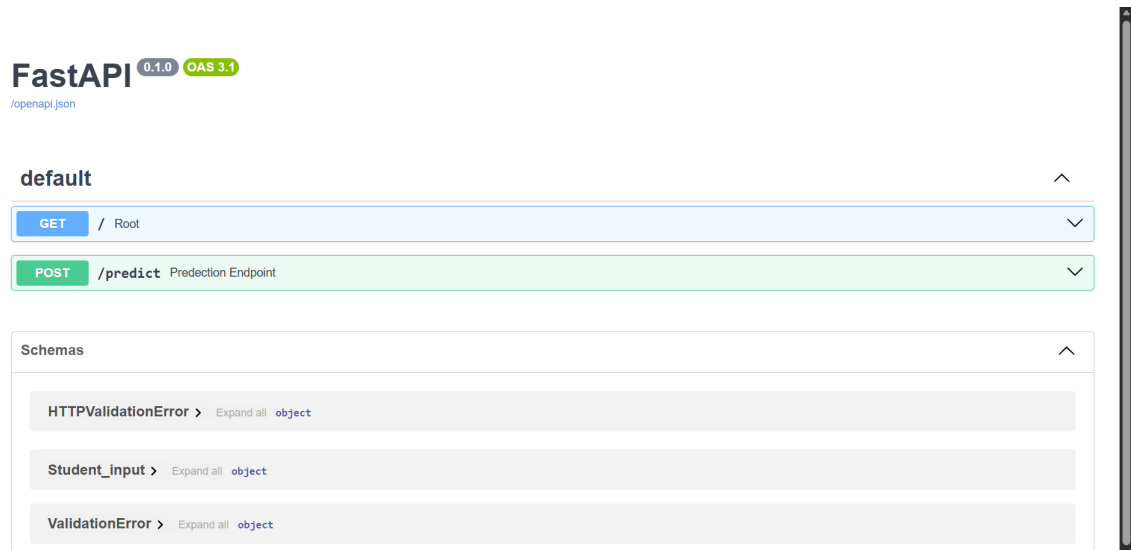


FIGURE 4.5 – Interface Swagger UI de l'API listant les endpoints et les schémas de données associés

L'exécution interactive d'une requête de test via cette interface (Figure ??) confirme la robustesse du système et la rapidité du traitement de l'inférence. Elle valide également que toute requête incomplète est interceptée en amont par un code d'erreur 422 `Unprocessable Entity`.

POST /predict Prediction Endpoint

Parameters

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "age": 19,
  "sex": 0,
  "edu": 0,
  "edu": 0,
  "studytme": 3,
  "failures": 0,
  "absences": 4,
  "goout": 0,
  "work": 0,
  "health": 5,
  "high": 1,
  "internet": 1,
  "traveltime": 1,
  "freetime": 3,
  "famrel": 5
}
```

Execute

(a) Exécution de la requête d'inférence

Code Details

200

Response body

```
{
  "The result is ": 1
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:7860
content-length: 20
content-type: application/json
date: Tue, 02 Jun 2026 00:38:47 GMT
server: uvicorn
vary: Origin
```

Responses

Code	Description
200	Successful Response

Media type application/json

(b) Charge utile JSON renvoyée

FIGURE 4.6 – Exécution d'un test d'inférence réussi avec retour de la charge utile JSON via l'interface Swagger

4.4 Conteneurisation de l'API avec Docker

Les conteneurs résolvent le problème des différences traditionnelles dans les environnements d'exécution. Ils permettent d'emballer l'application backend, les dépendances Python et le fichier du module du programme dans une image isolée et stable.

4.4.1 Dockerfile

Un fichier d'instruction `Dockerfile` a été placé à la racine du projet backend. Il spécifie les étapes séquentielles de construction de l'environnement virtuel :

```

api > Dockerfile
1 FROM python:3.11
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 EXPOSE 7860
12
13 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "7860"]

```

FIGURE 4.7 – Script de configuration Dockerfile pour la conteneurisation et le déploiement de l’API FastAPI.

4.4.2 Construction et exécution de l’image

La compilation des différentes couches logicielles est activée par la commande `docker build`. Ce processus traduit l’image binaire autonome et la stocke dans le registre local de l’hôte, comme montré par la figure 4.8.

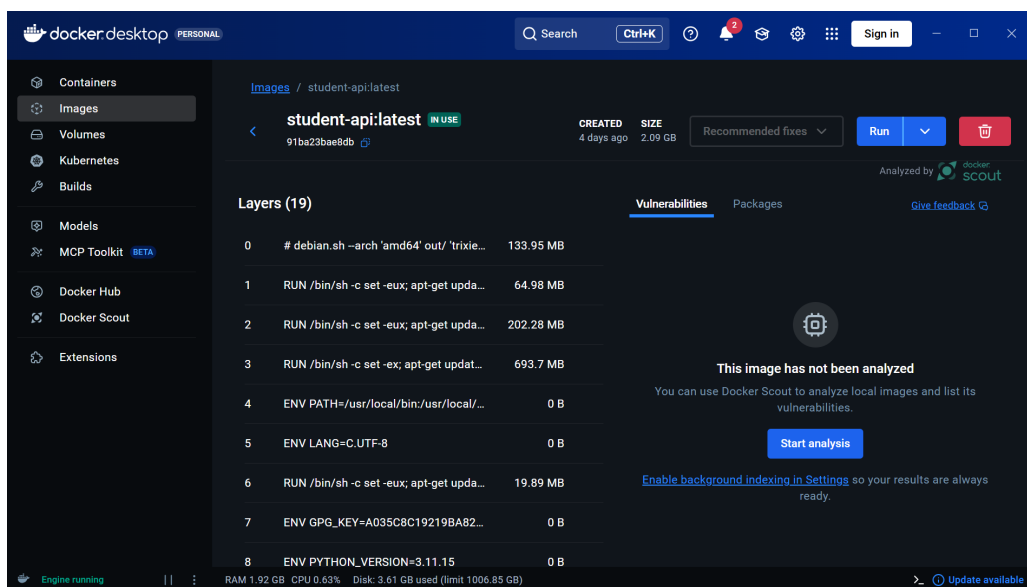


FIGURE 4.8 – Registre local des images Docker et build de la solution d’inférence

Une fois validée localement, le conteneur est exécuté en arrière-plan. La supervision de son cycle de vie, le suivi des allocations de ressources ainsi que la télémétrie des logs applicatifs en temps réel sont réalisés à l’aide de l’interface Docker Desktop (Figure 4.9).

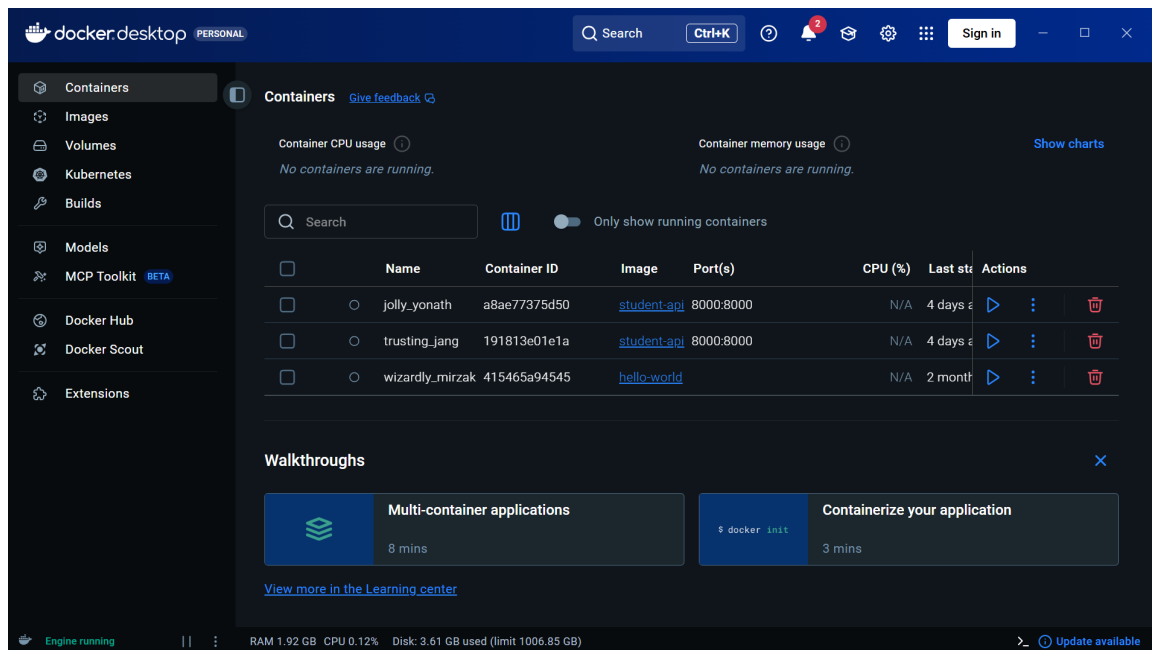


FIGURE 4.9 – Supervision du conteneur actif et télémétrie des logs via Docker Desktop

4.5 Déploiement de l'API sur Hugging Face Spaces

Pour rendre le service prédictif accessible de manière permanente par des applications tierces, l'image de l'API a été hébergée sur l'infrastructure cloud **Hugging Face Spaces**.

Ce déploiement a été configuré en sélectionnant un environnement d'exécution basé sur Docker. L'initialisation d'un dépôt Git distant connecté à Hugging Face déclenche, à chaque action de mise à jour (`git push`), un pipeline d'intégration et de déploiement continu (CI/CD) interne à la plateforme. La figure 4.10 présente la vue globale du tableau de bord utilisateur sur Hugging Face, validant l'état actif et la bonne santé réseau du Space dédié à l'API.

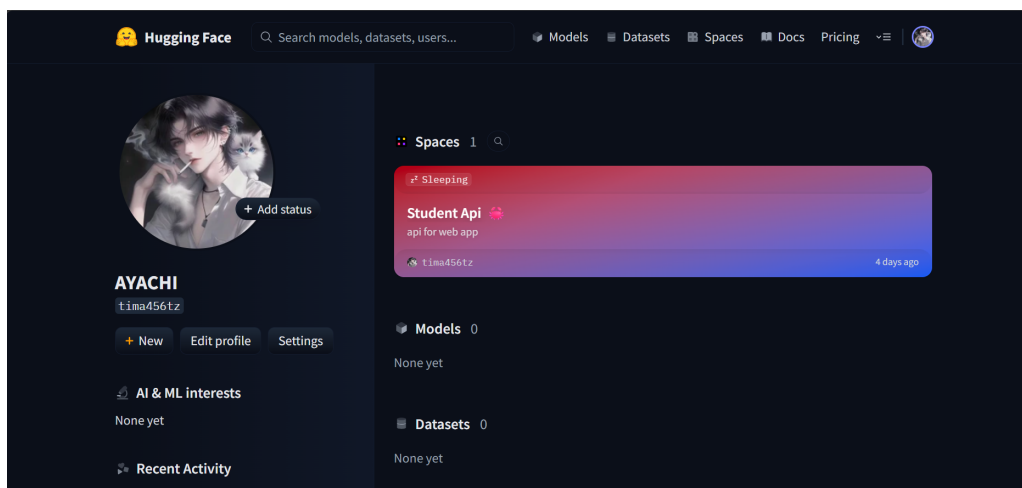


FIGURE 4.10 – Tableau de bord utilisateur Hugging Face et statut du Space de l’API

L’arborescence des fichiers sources déployés au sein du conteneur distant est directement synchronisée et inspectable sur la plateforme. Comme le met en évidence la figure 4.11, on y retrouve l’intégralité des briques logicielles indispensables : le script d’application FastAPI, le fichier d’instructions de build, la configuration des packages ainsi que l’artefact sérialisé du modèle champion.

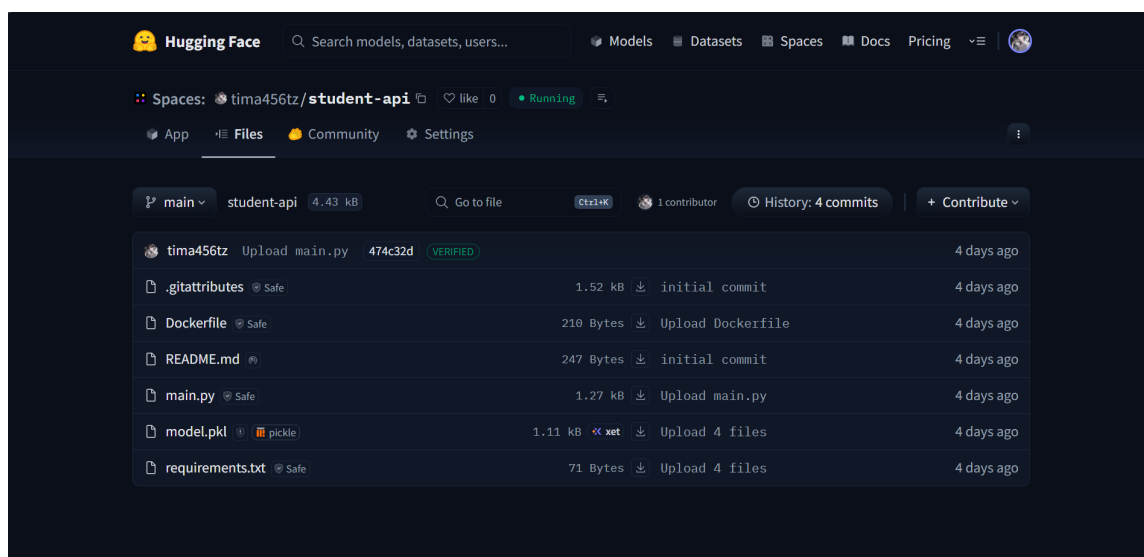


FIGURE 4.11 – Structure du référentiel de fichiers et des artefacts au sein du Space Hugging Face

4.6 Conception de la couche cliente avec Streamlit

L’exploitation d’une solution d’intelligence artificielle en milieu éducatif nécessite une interface d’utilisation accessible aux utilisateurs non techniciens. À

cette fin, une application web interactive a été conçue à l'aide du framework Streamlit.

4.6.1 Interface utilisateur et formulaire de saisie

L'interface graphique est développée en pur Python à l'aide de composants visuels standardisés. Pour collecter les 15 caractéristiques obligatoires requises par le modèle sans surcharger l'écran, les composants de formulaire de Streamlit ont été judicieusement répartis. Des menus déroulants (`st.selectbox`) et des curseurs numériques (`st.slider`) contraignent la saisie aux plages de valeurs exactes du dictionnaire des variables (Figure 4.12).

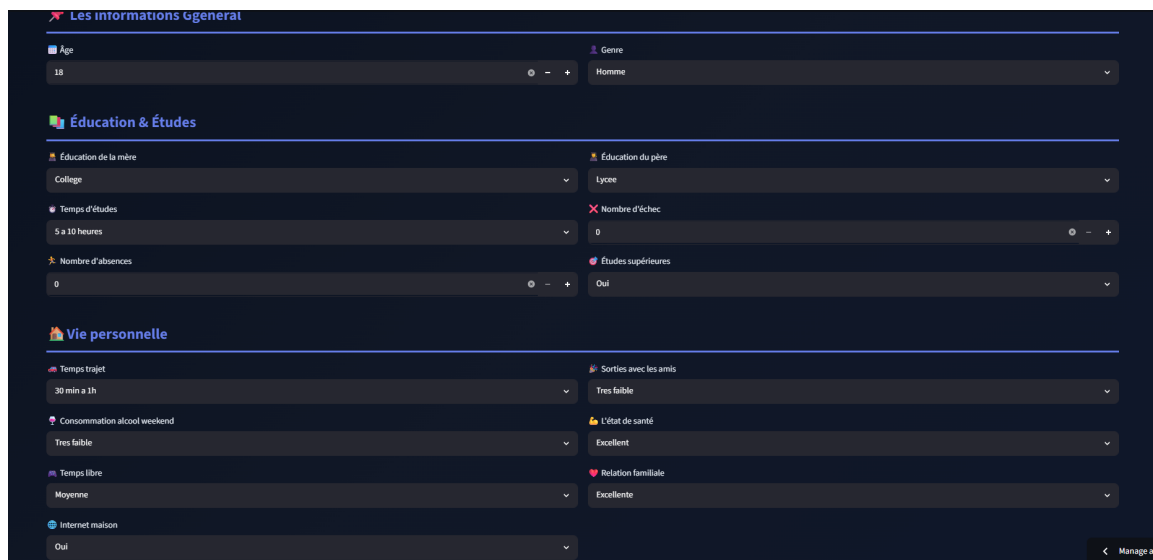
The image shows a Streamlit web application titled "Les informations générales" (General information). It features a dark theme and is organized into three main sections: "Éducation & Études" (Education & Studies), "Vie personnelle" (Personal life), and "Général" (General). Each section contains several interactive components like dropdown menus and sliders. The "Général" section includes fields for Age (18), Genre (Homme), and Temps trajet (30 min à 1h). The "Éducation & Études" section includes fields for Education de la mère (Collège), Education du père (Lycée), Temps d'études (5 à 10 heures), Nombre d'absences (0), Nombre d'échec (0), and Études supérieures (Oui). The "Vie personnelle" section includes fields for Consommation alcool weekend (Très faible), Sorties avec les amis (Très faible), L'état de santé (Excellent), and Relation familiale (Excellente). A "Manage app" button is visible in the bottom right corner.

FIGURE 4.12 – Formulaire Streamlit de saisie des caractéristiques étudiantes

4.6.2 Affichage des résultats

Une fois que l'utilisateur approuve le formulaire en cliquant sur le bouton d'envoi, l'application exécute la routine de connexion réseau. Lorsque la réponse du serveur backend est reçue, le résultat est décrypté et affiché de manière claire et dynamique. La figure 4.13 montre le rendu final de l'interface en production lors de l'affichage d'un profil étudiant analysé.

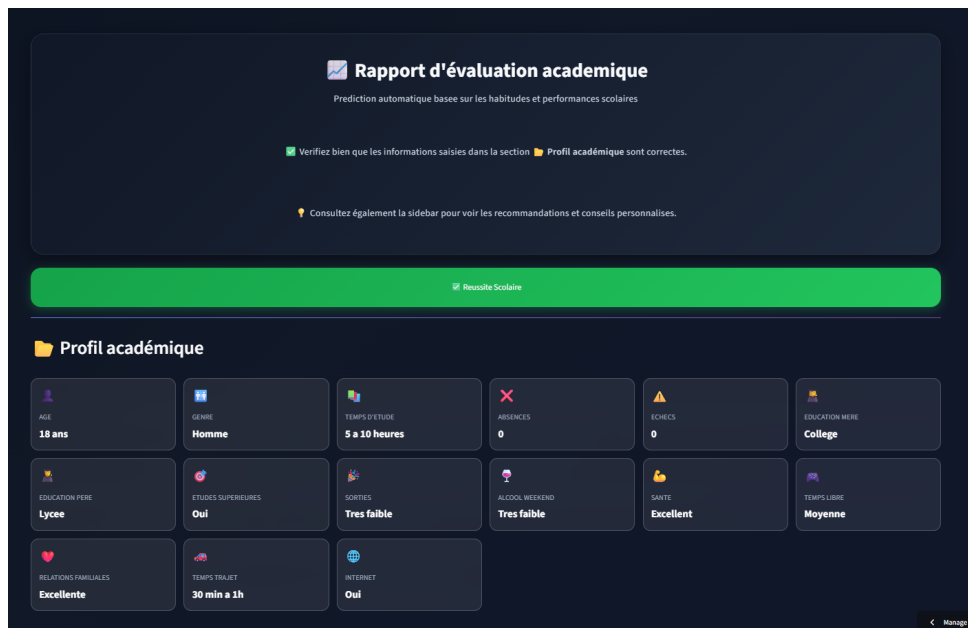


FIGURE 4.13 – Affichage de l'analyse prédictive finale sur l'interface de production Streamlit

4.7 Déploiement du Frontend sur Streamlit Community Cloud

Afin de fournir un accès public complet aux applications web sans imposer l'installation locale de l'écosystème sur les appareils des clients, la couche frontend a été déployée sur la plateforme SaaS *Streamlit Community Cloud*.

Publication et synchronisation directe avec le dépôt de code source public hébergé sur GitHub. Lorsqu'on connecte le projet, la plateforme Streamlit crée automatiquement un serveur d'application, installe le minimum de dépendances graphiques spécifiées dans le fichier de configuration, et génère un lien public sécurisé. Ce système assure une synchronisation en temps réel : toute modification sur la branche principale du dépôt GitHub se reflète immédiatement sur l'interface en ligne.

4.8 Connexion entre Streamlit et FastAPI

La cohésion et le travail synchronisé de notre architecture découpée dépendent du protocole d'échange de données sur le réseau via des requêtes HTTP REST. La mécanique interne d'une prédiction complète suit un cycle de quatre étapes séquentielles : capture et sérialisation des entrées utilisateur, requête ré-

seau sortante via la bibliothèque `requests`, inférence sur l'API FastAPI distante, et enfin désérialisation du JSON pour adapter l'affichage.

La figure 4.14 expose de manière comparative la vue d'ensemble du parcours utilisateur sur l'application web finale en ligne, depuis la complétion du formulaire de saisie jusqu'à la restitution graphique du verdict du modèle.

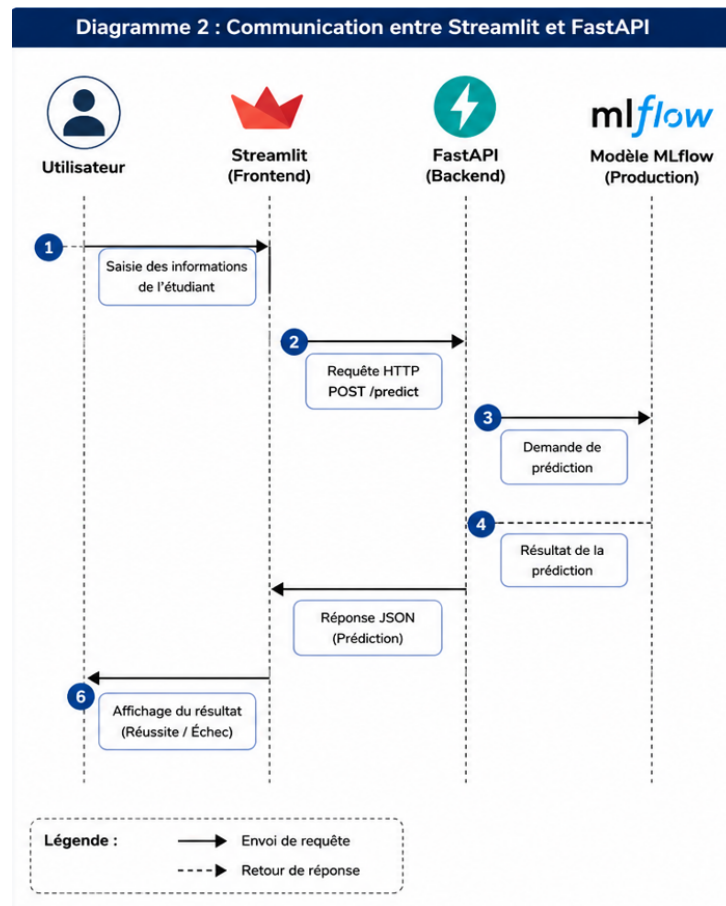


FIGURE 4.14 – Flux de communication entre Streamlit et FastAPI

Cette architecture distribuée isole totalement l'interface utilisateur de la logique de calcul mathématique, optimisant la vitesse d'exécution et facilitant la maintenance indépendante de chaque composant.

4.9 Tests finaux de la solution

Pour conclure le cycle d'industrialisation, des campagnes de tests d'intégration de bout en bout (*End-to-End Testing*) ont été menées en conditions réelles de production pour éprouver la robustesse de la solution cloud combinée.

Ces tests ont validé le comportement nominal du système à travers quatre scénarios de contrôle :

- **Disponibilité des services distants** : Validation de la communication permanente entre l'instance Streamlit Community Cloud et les conteneurs de Hugging Face Spaces.
- **Résilience face aux valeurs limites** : Injection de profils d'étudiants présentant des caractéristiques extrêmes (ex : maximum d'absences) afin de valider la stabilité du typage et l'absence de rupture de service.
- **Exactitude des prédictions en ligne** : Vérification stricte que les prédictions renvoyées par l'API cloud pour des profils types sont mathématiquement identiques aux résultats d'inférence obtenus localement lors de la phase d'évaluation de Scikit-Learn.
- **Mesure de performance de la chaîne de communication** : Évaluation du temps total d'attente utilisateur (*Round-Trip Time*). Le délai global reste inférieur à la seconde, confirmant la viabilité opérationnelle de l'application pour une utilisation quotidienne en milieu scolaire.

4.10 Monitoring, Observabilité et Workflow MLOps

En dehors de la vérification fonctionnelle, la continuité de la résolution de l'intelligence artificielle en environnement de production nécessite une surveillance active. Pour garantir la fiabilité de la solution et sa disponibilité continue, j'ai adopté une approche basée sur les standards MLOps en intégrant des outils de suivi et de surveillance :

- **Observabilité (Sentry)** : Conformément aux bonnes pratiques de la documentation officielle [15], J'ai intégré le SDK pour capturer les erreurs en temps réel. Cette méthode permet une analyse contextuelle du cycle de vie des erreurs.
- **Monitoring de disponibilité (UptimeRobot)** : En suivant les principes de disponibilité décrits par les standards de Hugging Face Spaces [12],

j’ai mis en place un système de monitoring actif assurant une disponibilité constante.

- **Pipeline de déploiement (CI/CD) :** Le workflow repose sur le déploiement continu automatisé, pratique recommandée pour les applications ML déployées sur le Hub [12].

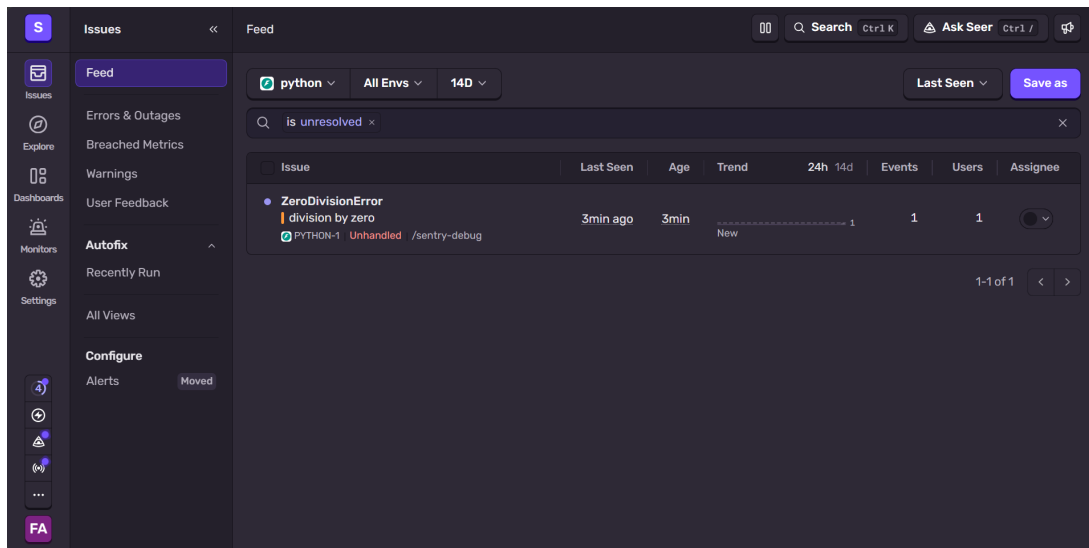


FIGURE 4.15 – Dashboard Sentry affichant la capture d’une erreur en temps réel.

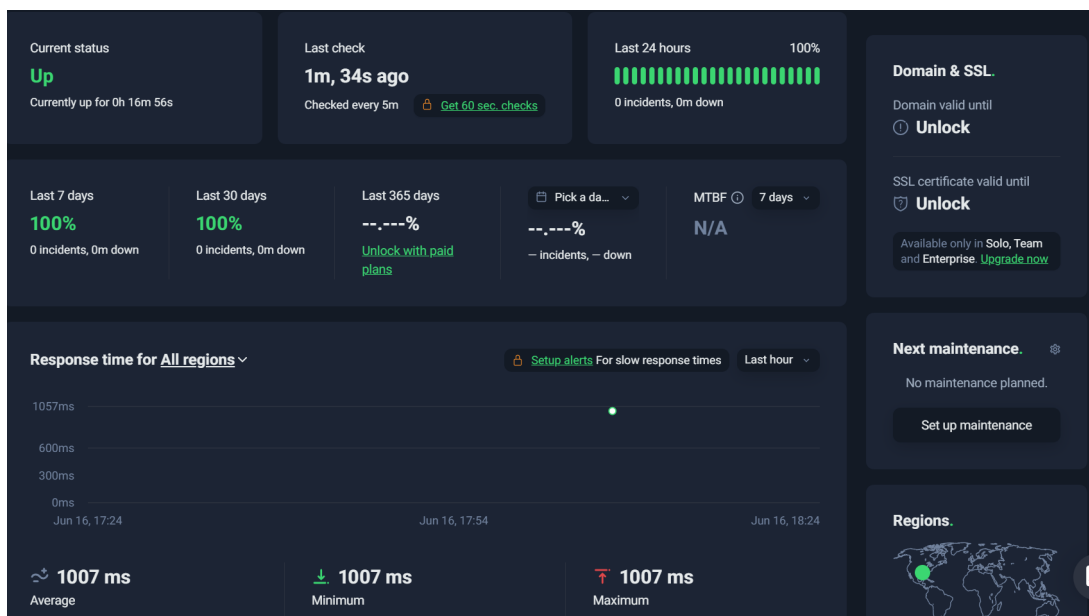


FIGURE 4.16 – Monitoring de disponibilité (Uptime 100%) via UptimeRobot.

Grâce à cette couche d’observabilité, la maintenance ne repose plus sur une réaction manuelle, mais sur une gestion proactive, assurant une stabilité technique conforme aux exigences industrielles.

4.11 Conclusion

Ce dernier chapitre était pour moi le plus challengeant du projet. Déployer une API sur Hugging Face Spaces et la connecter à une interface Streamlit hébergée ailleurs n'était pas évident au départ. J'ai rencontré des problèmes de CORS et de timeout que j'ai dû résoudre avant que la communication entre les deux services fonctionne correctement. Au final, la solution est accessible depuis n'importe quel navigateur, ce qui était l'objectif principal.



CONCLUSION GÉNÉRALE

Le présent projet de fin d'études a consisté en la conception, le développement et le déploiement d'une solution MLOps complète dédiée à la prédiction précoce de la réussite et du décrochage des étudiants. En combinant les techniques avancées de l'ingénierie des données, la puissance statistique du Machine Learning et la rigueur opérationnelle des architectures de production, nous avons mis en place un système capable d'accompagner efficacement les équipes pédagogiques dans leurs prises de décision.

Bilan du projet

D'un point de vue technique et méthodologique, les objectifs initiaux de ce travail ont été entièrement atteints à travers plusieurs étapes principales :

- **Ingénierie des données maîtrisée** : la phase d'exploration et de prétraitement a permis de nettoyer le jeu de données, de gérer les valeurs manquantes et de coder les variables comportementales et démographiques de manière idéale.
- **Performance et robustesse algorithmique** : a comparaison entre plusieurs structures de classification a montré la supériorité de la régression logistique comme modèle principal, avec une précision (Accuracy) de 75,95% et un score F1 de 81,55%, offrant un excellent compromis entre détection des profils à risque et limitation des faux diagnostics.
- **Gouvernance MLOps complète** : L'intégration de la plateforme *MLflow* a industrialisé le processus d'expérimentation en assurant un tracking im-

muable des hyperparamètres, la journalisation automatique via *Autolog*, et une gestion centralisée du cycle de vie des modèles au sein du *Model Registry*.

- **Industrialisation et accessibilité cloud** : Grâce à la conteneurisation sous *Docker*, l'encapsulation du modèle via une API REST avec *FastAPI* (déployée sur *Hugging Face Spaces*) et la création d'une interface utilisateur dynamique avec *Streamlit* (déployée sur *Streamlit Community Cloud*), la solution est aujourd'hui totalement opérationnelle, isolée et accessible publiquement via un simple navigateur internet.

Limites de la solution

Bien que d'excellents résultats aient été obtenus, plusieurs limites et domaines d'amélioration ont été identifiés lors des phases de test :

- **Sensibilité au déséquilibre des classes** : bien que cela soit compensé en ajustant les poids pendant l'entraînement, le déséquilibre initial dans le jeu de données limite mécaniquement la capacité du modèle à généraliser parfaitement sur les fichiers rares pendant les phases de test.
- **Volatilité des données d'entrée** : L'inférence repose sur des variables figées à un instant T . Le modèle ne prend pas en compte l'évolution dynamique et temporelle du comportement de l'étudiant (comme une baisse soudaine de présence en milieu de semestre).
- **Dépendance aux infrastructures cloud gratuites** : L'hébergement actuel de l'API et du frontend repose sur des instances mutualisées à ressources limitées, ce qui pourrait engendrer des latences ou des mises en veille du conteneur en cas de fortes charges de requêtes simultanées.

Perspectives

Pour maintenir et développer cette plateforme, il y a trois principaux axes de développement envisagés pour l'avenir :

1. **Intégration de modèles plus complexes (Ensemble Learning) :** introduire des algorithmes non linéaires plus robustes, comme *les forêts aléatoires* ou *XGBoost*, combinés à une recherche plus large des hyperparamètres, pourrait améliorer considérablement la capacité discriminante du moteur de prédiction.
2. **Mise en place d'un pipeline de réentraînement continu (Continuous Retraining) :** Pour contrer le phénomène de dérive des données (*Data Drift*) et de dérive du modèle (*Concept Drift*), il serait pertinent d'automatiser le réentraînement périodique du modèle champion via des outils d'orchestration de workflows comme *Apache Airflow* ou *Prefect*, dès que de nouvelles données d'étudiants sont validées.
3. **Interprétabilité locale des prédictions (XAI) :** L'intégration d'outils d'IA explicable (comme les valeurs *SHAP* ou *LIME*) au sein de l'interface Streamlit permettrait de ne pas fournir seulement une prédiction brute, mais d'expliquer textuellement et graphiquement aux équipes pédagogiques *pourquoi* tel étudiant est jugé à risque (ex : "Risque d'échec à 80% en raison d'un taux d'absence élevé et d'un faible temps de connexion").

En conclusion, ce projet de fin d'études m'a permis de consolider mes compétences transversales en **Data Engineering, Machine Learning** et **MLOps**. Il démontre concrètement l'apport des technologies de l'intelligence artificielle appliquée à l'éducation, ouvrant la voie à une pédagogie prédictive, plus inclusive et résolument tournée vers l'accompagnement personnalisé et la réussite des étudiants.

Bibliographie

- [1] IBM. What is artificial intelligence (ai) ?, 2024. Consulté en juin 2026.
- [2] IBM. What is machine learning ?, 2024. Consulté en juin 2026.
- [3] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. O'Reilly Media, 3rd edition, 2022.
- [4] Scikit-learn Developers. Scikit-learn documentation, 2024. Consulté en juin 2026.
- [5] Amazon Web Services. What is mlops ?, 2024. Consulté en juin 2026.
- [6] Google Cloud. Mlops : Continuous delivery and automation pipelines in machine learning, 2024. Consulté en juin 2026.
- [7] Dominik Kreuzberger, Niklas Köhl, and Sebastian Hirschl. Machine learning operations (mlops) : Overview, definition, and architecture. *IEEE Access*, 11 :31866–31879, 2023.
- [8] MLflow. MLflow documentation, 2024. Consulté en juin 2026.
- [9] FastAPI. Fastapi documentation, 2024. Consulté en juin 2026.
- [10] Streamlit. Streamlit documentation, 2024. Consulté en juin 2026.
- [11] Docker. Docker documentation, 2024. Consulté en juin 2026.
- [12] Hugging Face. Hugging face spaces documentation, 2024. Consulté en juin 2026.
- [13] Paulo Cortez and Alice Maria Gonçalves Silva. Using data mining to predict secondary school student performance. *Proceedings of 5th Future Business Technology Conference*, pages 5–12, 2008.
- [14] Paulo Cortez. Student Performance Dataset. UCI Machine Learning Repository, 2008. DOI : <https://doi.org/10.24432/C5TG7T>.

- [15] Sentry. Sentry documentation : Error tracking observability, 2026.
Consulté en juin 2026.